



Machine Learning 1

Lecture 5 - Classification

Recap/Catch-up:

Bias-Variance

Bayesian Predictive Distributions

Equivalent Kernel (later, wk6)

Continue with:

Bayesian Model Comparison

Empirical Bayes

Classification

Decision Theory

Generative Models

Erik Bekkers



The probabilistic view



W7

Gaussian Processes

W6

Kernel Methods

W5

Latent variable models

W4

Neural Networks

W3

Classification

W2

Regression

W1

Probabilistic modeling

Random variables, distributions,
maximum likelihood estimation

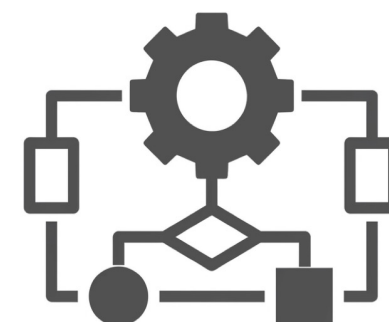
Optimization

Exact/analytic, (stochastic
gradient descent, constrained
optimization, 2nd order)

Mathematical tools

Multi-variate calculus, linear
algebra

The algorithmic view





Machine Learning 1

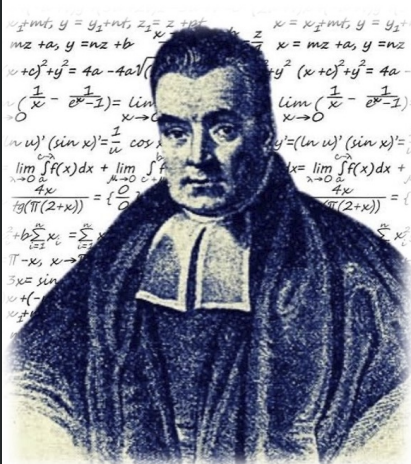
Lecture 4.2 - Supervised Learning

Bias Variance Decomposition

Erik Bekkers

(Bishop 1.5.5, 3.2)

The probabilistic view



W1

Probabilistic modeling

Random variables, distributions,
maximum likelihood estimation,
MAP

W7

Gaussian Processes

W6

Kernel Methods

W5

Latent variable models

W4

Neural Networks

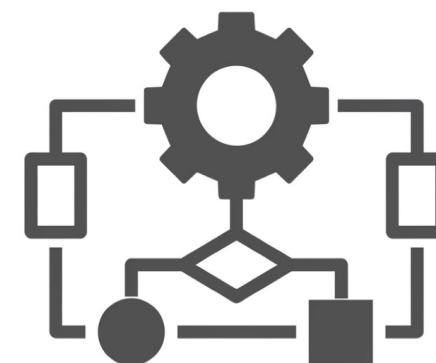
W3

Classification

W2

Regression

The algorithmic view



Expected Loss for Regression $\mathbb{E}_{(\mathbf{x}, t) \sim p(\mathbf{x}, t)}[L(t, y(\mathbf{x}))]$

Why do models make errors?

What kind of errors can we expect?

- Consider dataset of observations $(\mathbf{x}, t) \sim p(\mathbf{x}, t)$ and a model $y(\mathbf{x})$

$$D = \{(x_1, t_1), (x_2, t_2), \dots\}, 5 \rightarrow y_0$$

- The model makes errors (regression loss function):

$$L(t, y(\mathbf{x})) = (t - \hat{t})^2$$

- Every time we make an observation of random variables $(\mathbf{x}, t)$ we make a different error. We now consider the expected loss:

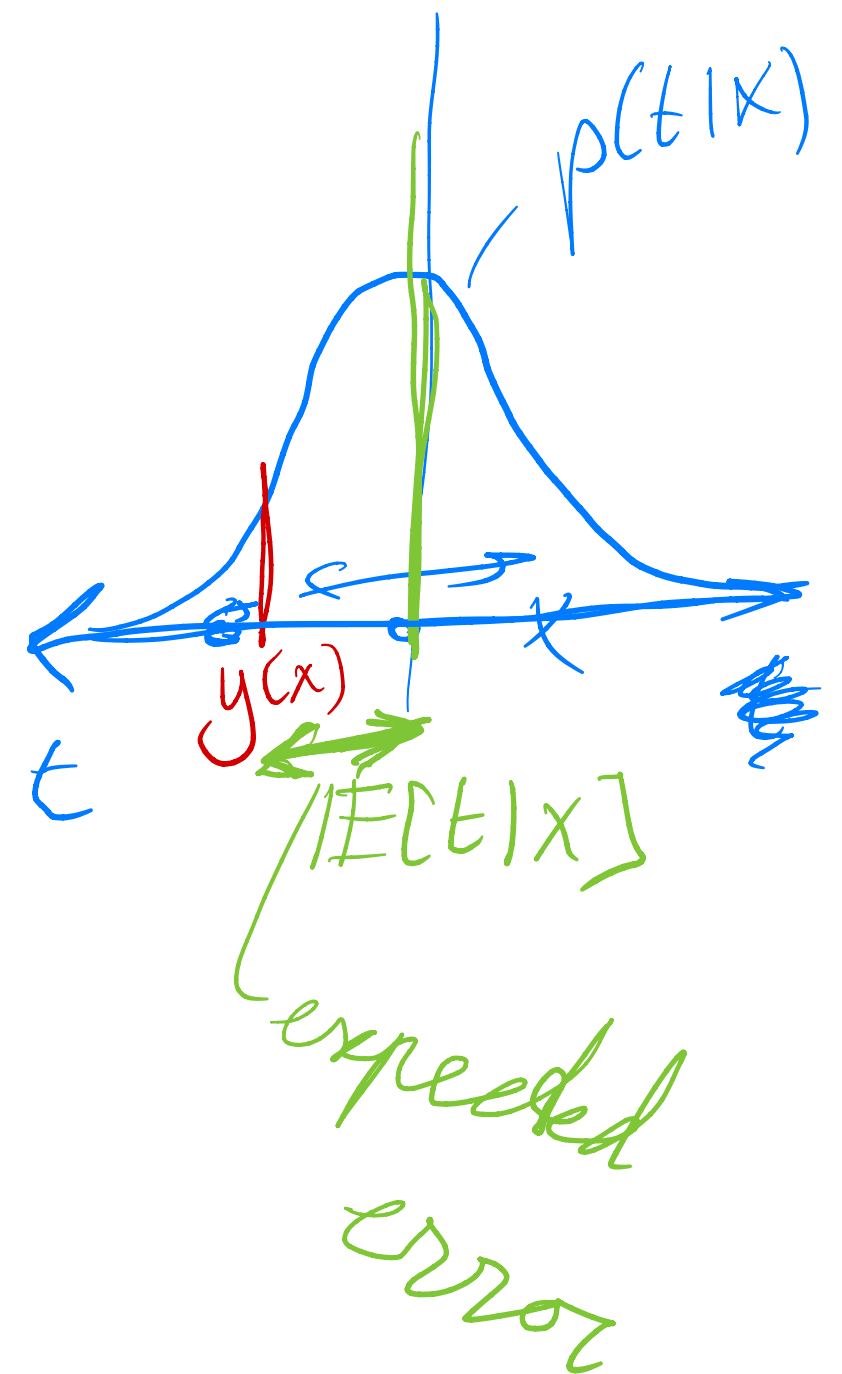
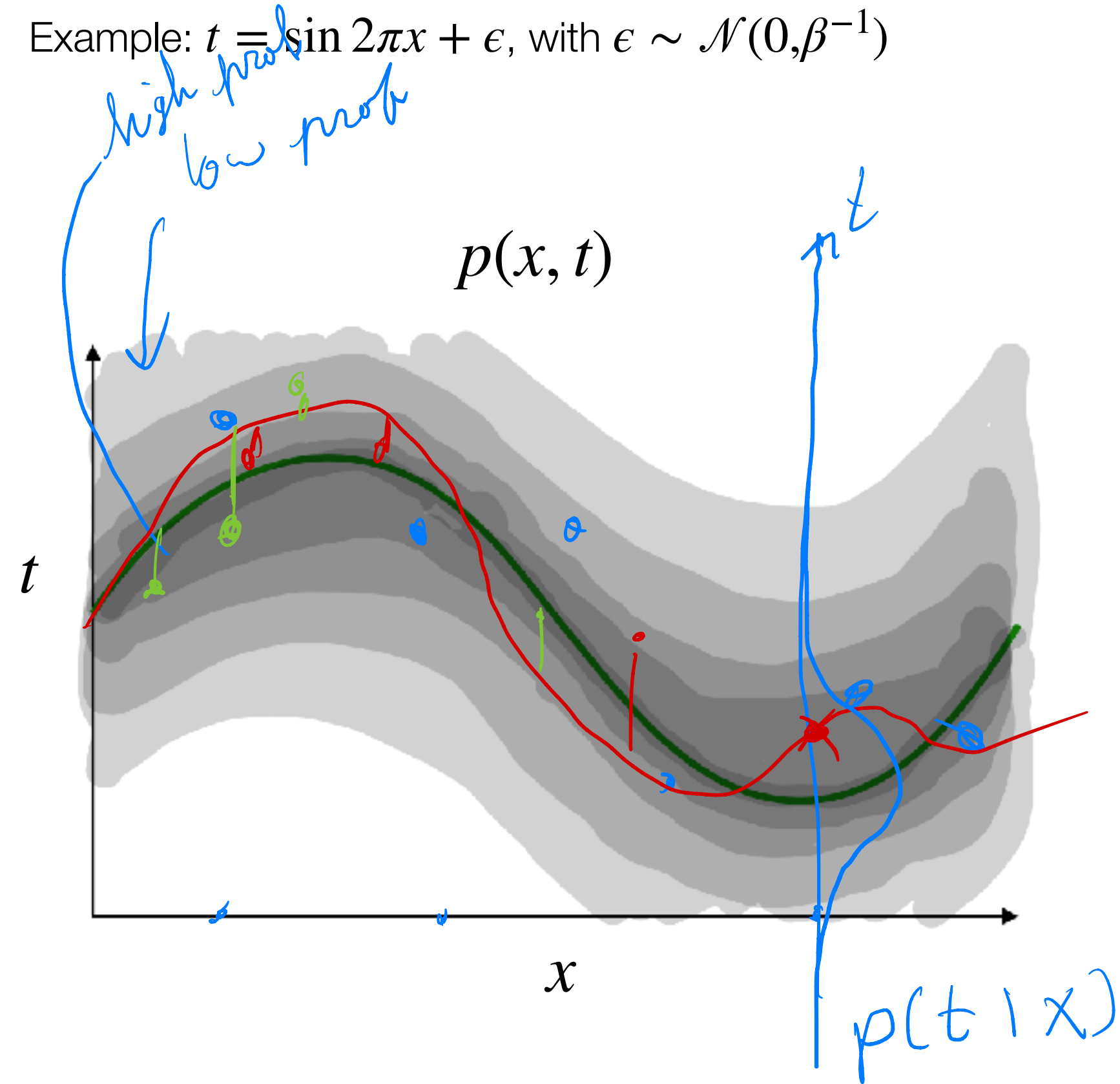
$$\mathbb{E}_{(\mathbf{x}, t) \sim p(\mathbf{x}, t)}[L(t, y(\mathbf{x}))] = \iint (t - y(\mathbf{x}))^2 p(\mathbf{x}, t) d\mathbf{x} dt$$

- The best model y we can possibly have (Bishop 1.5.5):

$$y(\mathbf{x}) = \mathbb{E}[t | \mathbf{x}]$$

Regression function

Example: $t = \sin 2\pi x + \epsilon$, with $\epsilon \sim \mathcal{N}(0, \beta^{-1})$



Expected Loss for Regression $\mathbb{E}_{(\mathbf{x},t) \sim p(\mathbf{x},t)}[L(t, y(\mathbf{x}))]$

- Consider the expected loss:

$$\mathbb{E}[L] = \iint (y(\mathbf{x}) - t)^2 p(\mathbf{x}, t) dt d\mathbf{x}$$

- Let's analyze it relative to the regression function $\mathbb{E}[t | \mathbf{x}] := \mathbb{E}_{t \sim p(t|\mathbf{x})}[t | \mathbf{x}]$:

$$\mathbb{E}[L] = \iint (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}] + \mathbb{E}[t | \mathbf{x}] - t)^2 p(\mathbf{x}, t) dt d\mathbf{x}$$

$a^2 + 2ab + b^2$

Expected Loss for Regression $\mathbb{E}_{(\mathbf{x}, t) \sim p(\mathbf{x}, t)}[L(t, y(\mathbf{x}))]$

- Consider the expected loss:

$$\mathbb{E}[L] = \iint (y(\mathbf{x}) - t)^2 p(\mathbf{x}, t) dt d\mathbf{x}$$

- Let's analyze it relative to the regression function $\mathbb{E}[t | \mathbf{x}] := \mathbb{E}_{t \sim p(t|\mathbf{x})}[t | \mathbf{x}]$:

$$\mathbb{E}[L] = \iint (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}] + \mathbb{E}[t | \mathbf{x}] - t)^2 p(\mathbf{x}, t) dt d\mathbf{x}$$

(a + b)²

- Write out the square $((a + b)^2 = a^2 + 2ab + b^2)$ and use product rule ($p(\mathbf{x}, t) = p(t | \mathbf{x})p(\mathbf{x})$)

$$\begin{aligned} \mathbb{E}[L] &= \iint (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2 p(t | \mathbf{x}) dt p(\mathbf{x}) d\mathbf{x} \\ &\quad + 2 \iint (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}]) (\mathbb{E}[t | \mathbf{x}] - t) p(t | \mathbf{x}) dt p(\mathbf{x}) d\mathbf{x} \\ &\quad + \iint (\mathbb{E}[t | \mathbf{x}] - t)^2 p(t | \mathbf{x}) dt p(\mathbf{x}) d\mathbf{x} \end{aligned}$$

① $\int t p(t | \mathbf{x}) dt = \mathbb{E}[t | \mathbf{x}]$

② $\text{var}[t | \mathbf{x}]$

Why did that term vanish? ①

$$2) \int \int [y(x) - E[t|x]] (E[t|x] - t) p(t|x) dt$$

↳ does not depend on t
so we can move it out
of the integral

Let's focus on the integral:

$$\begin{aligned} & \int (E[t|x] - t) p(t|x) dt \\ &= \int E[t|x] p(t|x) dt - \int t p(t|x) dt \end{aligned}$$

Remember definition of expectation

$$= \int E[t|x] p(t|x) dt - E[t|x]$$

↳ is not a random variable and does not depend on t . The expectation is just a number, a constant, so can be moved out of the integral.

$$= E[t|x] \int p(t|x) dt - E[t|x]$$

↳ = 1 since all distributions integrate to 1 (normalization)

$$= E[t|x] - E[t|x] = 0$$

Where did the variance term (2) come from?

Remember: the variance is defined as

$$\text{Var}[t] = E[(t - E[t])^2]$$

(same as for conditional distribution)

So for

$$\int (E[t|x] - t)^2 p(t|x) dt$$

$$= E[(E[t|x] - t)^2 | x]$$

$$= \text{Var}[t|x]$$

Expected Loss for Regression $\mathbb{E}_{(\mathbf{x}, t) \sim p(\mathbf{x}, t)}[L(t, y(\mathbf{x}))]$

- Consider the expected loss:

$$\mathbb{E}[L] = \int \int (y(\mathbf{x}) - t)^2 p(\mathbf{x}, t) dt d\mathbf{x}$$

- Let's analyze it relative to the regression function $\mathbb{E}[t | \mathbf{x}] := \mathbb{E}_{t \sim p(t|\mathbf{x})}[t | \mathbf{x}]$:

$$\mathbb{E}[L] = \int \int (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}] + \mathbb{E}[t | \mathbf{x}] - t)^2 p(\mathbf{x}, t) dt d\mathbf{x}$$

- Write out the square $((a + b)^2 = a^2 + 2ab + b^2)$ and use product rule $(p(\mathbf{x}, t) = p(t | \mathbf{x})p(\mathbf{x}))$

$$\mathbb{E}[L] = \int \int (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2 p(t | \mathbf{x}) dt p(\mathbf{x}) d\mathbf{x} + \int \int (\mathbb{E}[t | \mathbf{x}] - t)^2 p(t | \mathbf{x}) dt p(\mathbf{x}) d\mathbf{x}$$

- Thus the expected loss can be decomposed in two terms

$$\mathbb{E}[L] = \int (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2 p(\mathbf{x}) d\mathbf{x} + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x}$$

sub-optimal
y

↓
inherent noise

has nothing to do with y

Summary so far...

- Any model y will make errors, this expected loss decomposes into

$$\mathbb{E}_{(\mathbf{x}, t) \sim p(\mathbf{x}, t)}[L(t, y(\mathbf{x}))] = \underbrace{\int (y(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2 p(\mathbf{x}) d\mathbf{x}}_{\text{zoom-in}} + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x}$$

- The best possible model is the regression function $y(\mathbf{x}) = \mathbb{E}[t | \mathbf{x}]$
given $D_1 = \{(x_1, t_1), \dots, (x_n, t_n)\} \mapsto y_{D_1}$
 $D_2 = \{(x_1, t_1), \dots, (x_n, t_n)\} \mapsto y_{D_2}$

- In practice we approximate it with fits $y_D = \operatorname{argmin}_y \sum_{(\mathbf{x}, t) \in D} L(t, y(\mathbf{x}))$
- What can we say about the expected loss of y_D ?

The Average Expected Loss

- Let's analyze performance of a learning algorithm by averaging the expected loss over learned y_D for different datasets D

$$\mathbb{E}_D[\mathbb{E}[L]] = \int \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] p(\mathbf{x}) d\mathbf{x} + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x}$$

The Average Expected Loss

- ▶ Let's analyze performance of a learning algorithm by averaging the expected loss over learned y_D for different datasets D

$$\mathbb{E}_D[\mathbb{E}[L]] = \int \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] p(\mathbf{x}) d\mathbf{x} + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x}$$

- ▶ Analyze it relative to the average model $\mathbb{E}_D[y_D(\mathbf{x})]$

$$\mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] = \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})] + \mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2]$$

The Average Expected Loss

- Let's analyze performance of a learning algorithm by averaging the expected loss over learned y_D for different datasets D

$$\mathbb{E}_D[\mathbb{E}[L]] = \int \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] p(\mathbf{x}) d\mathbf{x} + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x}$$

- Analyze it relative to the average model $\mathbb{E}_D[y_D(\mathbf{x})]$

$$\mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] = \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})] + \mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2]$$

(Expand square $(a + b)^2 = a^2 + 2ab + b^2$)

$$\begin{aligned} &= \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})])^2] + \mathbb{E}_D[(\mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2] \\ &\quad + 2 \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})])(\mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])] \end{aligned}$$

The Average Expected Loss

- Let's analyze performance of a learning algorithm by averaging the expected loss over learned y_D for different datasets D

$$\mathbb{E}_D[\mathbb{E}[L]] = \int \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] p(\mathbf{x}) d\mathbf{x} + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x}$$

- Analyze it relative to the average model $\mathbb{E}_D[y_D(\mathbf{x})]$

$$\mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}[t | \mathbf{x}])^2] = \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})] + \mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2]$$

(Expand square $(a - b)^2 = a^2 - 2ab + b^2$)

recognize
as
variance!

$$\begin{aligned} &= \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})])^2] + \mathbb{E}_D[(\mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2] \\ &\quad - 2 \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})])(\mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])] \end{aligned}$$

(Compute expectation, rewrite)

$$= \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})])^2] + (\mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2$$

Bias-Variance Decomposition

- ▶ What can we say about the expected loss of y_D ?
- ▶ On average (over datasets D) our model y_D will make three types of errors:

$$\begin{aligned}\mathbb{E}_D[\mathbb{E}[L]] = & \int \mathbb{E}_D[(y_D(\mathbf{x}) - \mathbb{E}_D[y_D(\mathbf{x})])^2] p(\mathbf{x}) d\mathbf{x} && \text{Variance} \\ & + \int (\mathbb{E}_D[y_D(\mathbf{x})] - \mathbb{E}[t | \mathbf{x}])^2 p(\mathbf{x}) d\mathbf{x} && \text{Bias}^2 \\ & + \int \text{var}[t | \mathbf{x}] p(\mathbf{x}) d\mathbf{x} && \text{Noise}\end{aligned}$$

Bias-Variance Decomposition: Example

- Generate $L = 100$ datasets of $N = 50$ points:

- $x \sim U(0,1)$
- $t = \sin(2\pi x) + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \alpha^{-1})$

$\mathbb{E}[t|x] = \sin 2\pi x$

- Parametrize y with 24 Gaussian basis functions

The $L = 100$ datasets each gives a model

$y^{(l)}(x) = (\mathbf{w}^{(l)})^T \boldsymbol{\phi}(x)$

- That minimizes:

$E_D = \frac{1}{2} \sum_{i=1}^N \{t_n - \mathbf{w}^T \boldsymbol{\phi}(x)\}^2 + \frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$

- Let $\bar{y}(x) = \mathbb{E}_D[y_D(x)]$ the average function

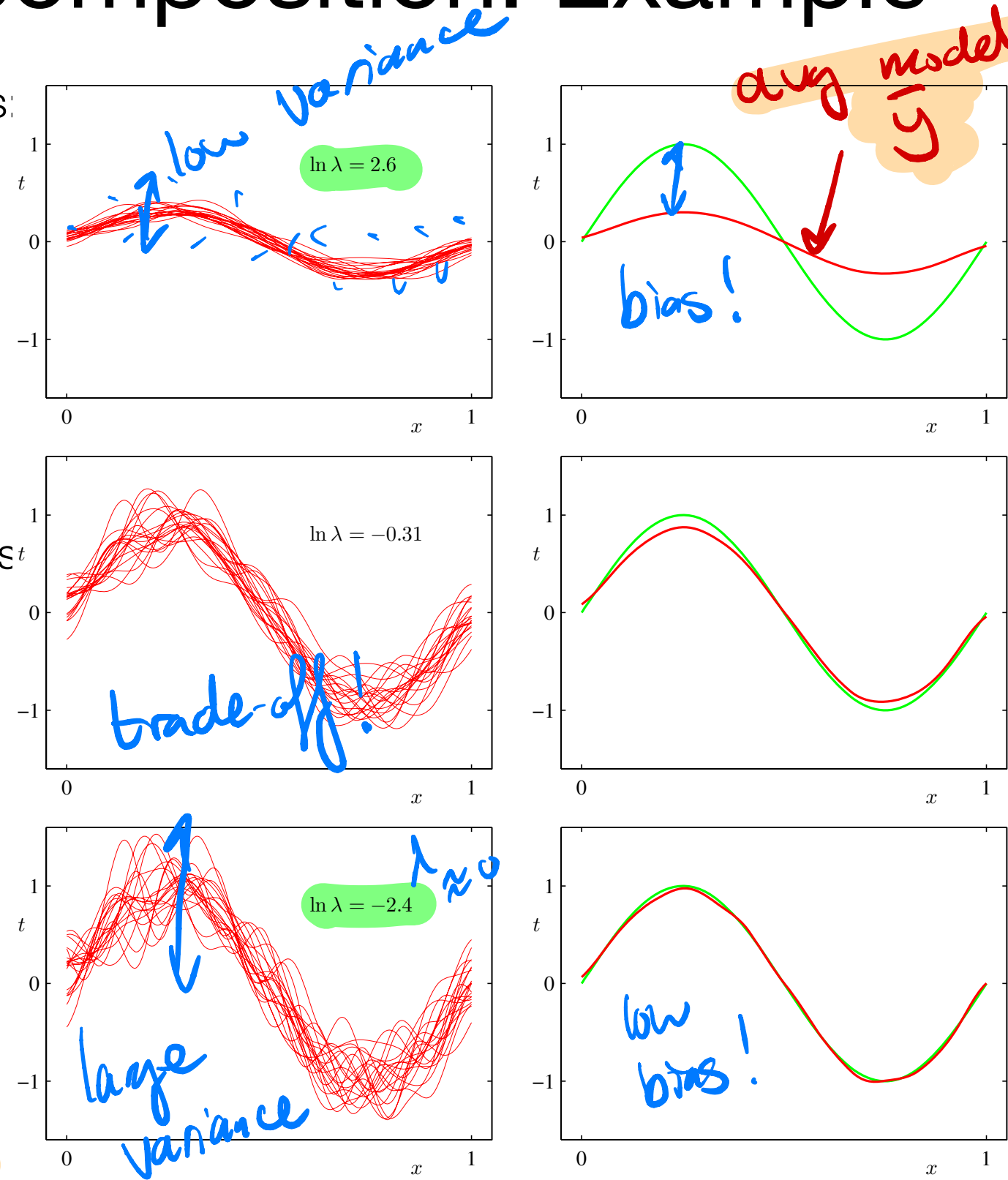


Figure: bias-variance decomposition (Bishop 3.5)

Bias-Variance Decomposition

- ▶ What can we say about the expected loss of y_D ?
- ▶ On average (over datasets D) our model y_D will make three types of errors:

$$\begin{aligned}\mathbb{E}_D[\mathbb{E}[L]] = & \int \mathbb{E}_D[(y_D(x) - \mathbb{E}_D[y_D(x)])^2] p(x) dx && \text{Variance} \\ & + \int (\mathbb{E}_D[y_D(x)] - \mathbb{E}[t | x])^2 p(x) dx && \text{Bias}^2 \\ & + \int \text{var}[t | x] p(x) dx && \text{Noise}\end{aligned}$$

Bias-Variance Decomposition

- ▶ What can we say about the expected loss of y_D ?
- ▶ On average (over datasets D) our model y_D will make three types of errors:

$$\mathbb{E}_D[\mathbb{E}[L]] \approx \frac{1}{N} \sum_{n=1}^N \frac{1}{L} \sum_{l=1}^L (\mathbf{y}^{(l)}(x_n) - \bar{\mathbf{y}}(\mathbf{x}_n))^2 \quad \text{Variance}$$

$$+ \frac{1}{N} \sum_{i=1}^N \bar{\mathbf{y}}(x_n) - \sin 2\pi x_n)^2 \quad \text{Bias}^2$$

see lecture 1
for finite dataset
approximation of expectation

Bias-Variance Decomposition: Example

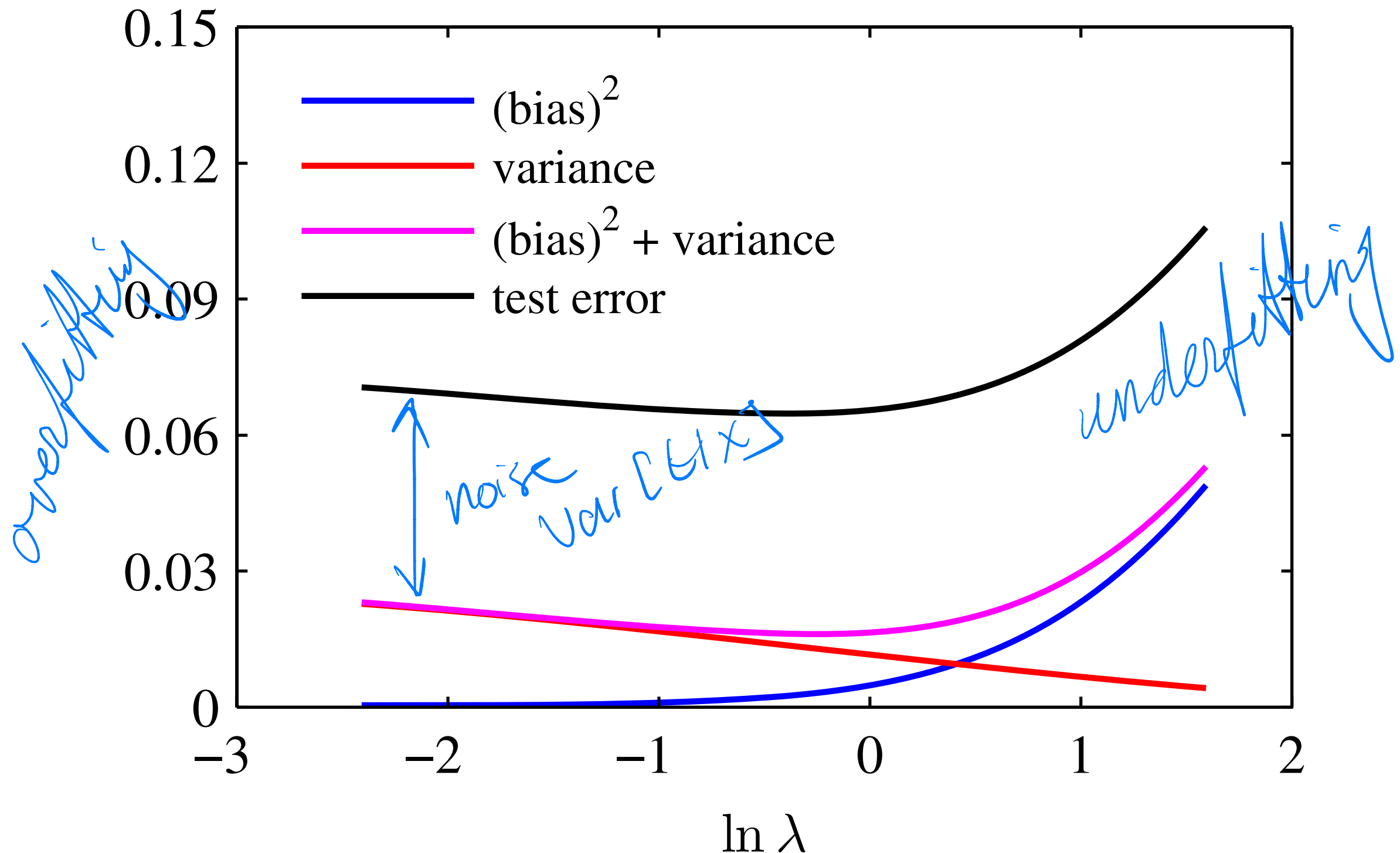


Figure: bias-variance decomposition (Bishop 3.6)

Bias-Variance Decomposition

- ▶ In practice we don't want to split our dataset into L datasets to determine the best model complexity (best value of λ)
- ▶ Better to keep large dataset
 - ▶ Less overfitting.
 - ▶ Different optimal model complexity!
- ▶ Model averaging? Bayesian regression!



Machine Learning 1

Lecture 4.3 - Supervised Learning
Bayesian Linear Regression - **Gaussian**
Posteriors

Erik Bekkers

(Bishop 3.3.1 (and 2.3.3))

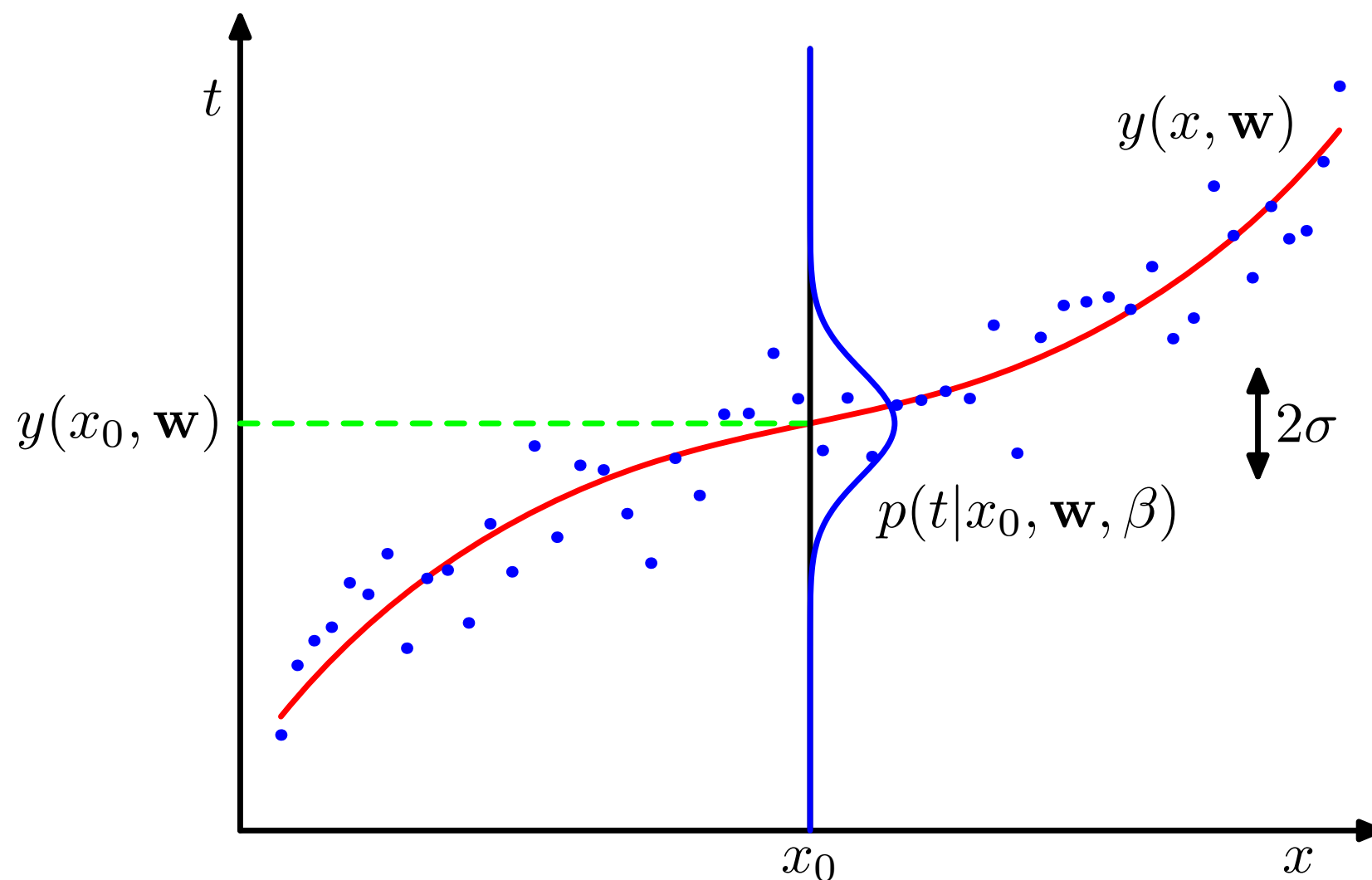


Bayesian Linear Regression

► Regression problem with:

► Data: $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, $\mathbf{t} = (t_1, \dots, t_N)^T$

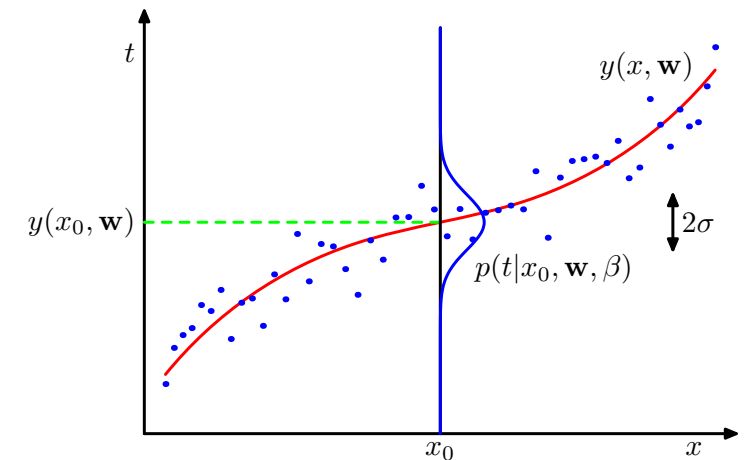
► Predictive distribution $p(t' | \mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t' | \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}'), \beta^{-1})$



Bayesian Linear Regression

- Regression problem with:

- Data: $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$, $\mathbf{t} = (t_1, \dots, t_N)^T$
- Predictive distribution $p(t' | \mathbf{x}, \mathbf{w}, \beta) = \mathcal{N}(t' | \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}'), \beta^{-1})$



- Probabilistic model with Gaussians:

- Likelihood:
$$p(\mathbf{t} | \mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n), \beta^{-1}) = \mathcal{N}(\mathbf{t} | \boldsymbol{\Phi} \mathbf{w}, \beta^{-1} \mathbf{I})$$

- Conjugate prior: $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{m}_0, \mathbf{S}_0)$

- Posterior:
$$p(\mathbf{w} | \mathbf{t}, \mathbf{X}) = \frac{p(\mathbf{t} | \mathbf{X}, \mathbf{w}, \beta) p(\mathbf{w})}{p(\mathbf{t} | \mathbf{X}, \beta)} = \mathcal{N}(\mathbf{w} | \mathbf{m}_N, \mathbf{S}_N)$$

Bishop Ch 2.3, Eq. 2.116

$$\mathbf{S}_N^{-1} = \mathbf{S}_0^{-1} + \beta \boldsymbol{\Phi}^T \boldsymbol{\Phi}$$

$$\mathbf{m}_N = \mathbf{S}_N (\mathbf{S}_0^{-1} \mathbf{m}_0 + \beta \boldsymbol{\Phi}^T \mathbf{t})$$

Bayesian Linear Regression

Regression problem with:

So, with a **Gaussian** likelihood and **Gaussian** prior

we obtain an

explicit expression for the **posterior**
(which is again **Gaussian**)



Probabilistic model with Gaussians:

Likelihood:
$$p(\mathbf{t} | \mathbf{X}, \mathbf{w}, \beta) = \prod_{n=1}^N \mathcal{N}(t_n | \mathbf{w}^T \boldsymbol{\phi}(\mathbf{x}_n), \beta^{-1}) = \mathcal{N}(\mathbf{t} | \boldsymbol{\Phi} \mathbf{w}, \beta^{-1} \mathbf{I})$$

Conjugate prior: $p(\mathbf{w}) = \mathcal{N}(\mathbf{w} | \mathbf{m}_0, \mathbf{S}_0)$

Posterior:
$$p(\mathbf{w} | \mathbf{t}, \mathbf{X}) = \frac{p(\mathbf{t} | \mathbf{X}, \mathbf{w}, \beta) p(\mathbf{w})}{p(\mathbf{t} | \mathbf{X}, \beta)} = \mathcal{N}(\mathbf{w} | \mathbf{m}_N, \mathbf{S}_N)$$



Machine Learning 1

Lecture 4.4 - Supervised Learning
Bayesian Linear Regression - **Sequential
Bayesian Learning**

Erik Bekkers

(Bishop 3.3.1)



Example: Sequential Bayesian Learning

- Data generated by

$$t = -0.3 + 0.5x + \epsilon$$

- Prior

$$p(\mathbf{w} | \alpha) = \mathcal{N}(\mathbf{w} | \mathbf{0}, \alpha^{-1} \mathbf{I})$$

- Sample 1 datapoint ★

$$(x_1, t_1)$$

- Likelihood

$$p(t_1 | x_1, \mathbf{w}, \beta) = \mathcal{N}(t_1 | w_0 + w_1 x_1, \beta^{-1})$$

- Posterior

$$p(\mathbf{w} | x_1, t_1, \alpha, \beta) \propto p(t_1 | x_1, \mathbf{w}, \beta) p(\mathbf{w} | \alpha)$$

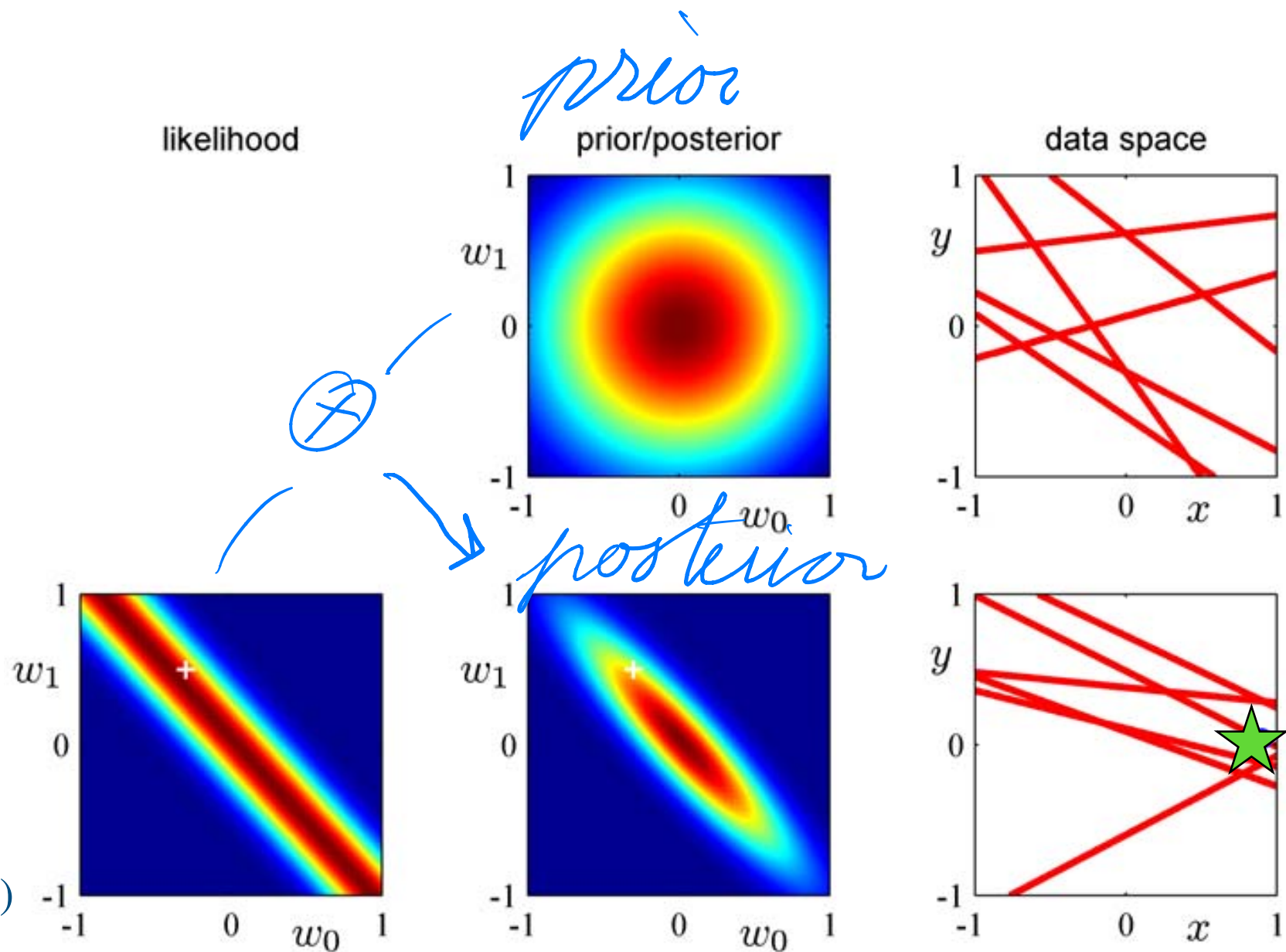


Figure: Sequential Bayesian learning (Bishop 3.7)

Example: Sequential Bayesian Learning

- ▶ Sample 2nd data point ★

- ▶ Posterior → Prior

- ▶ Likelihood

$$p(t_2 | x_2, \mathbf{w}, \beta)$$

- ▶ Posterior

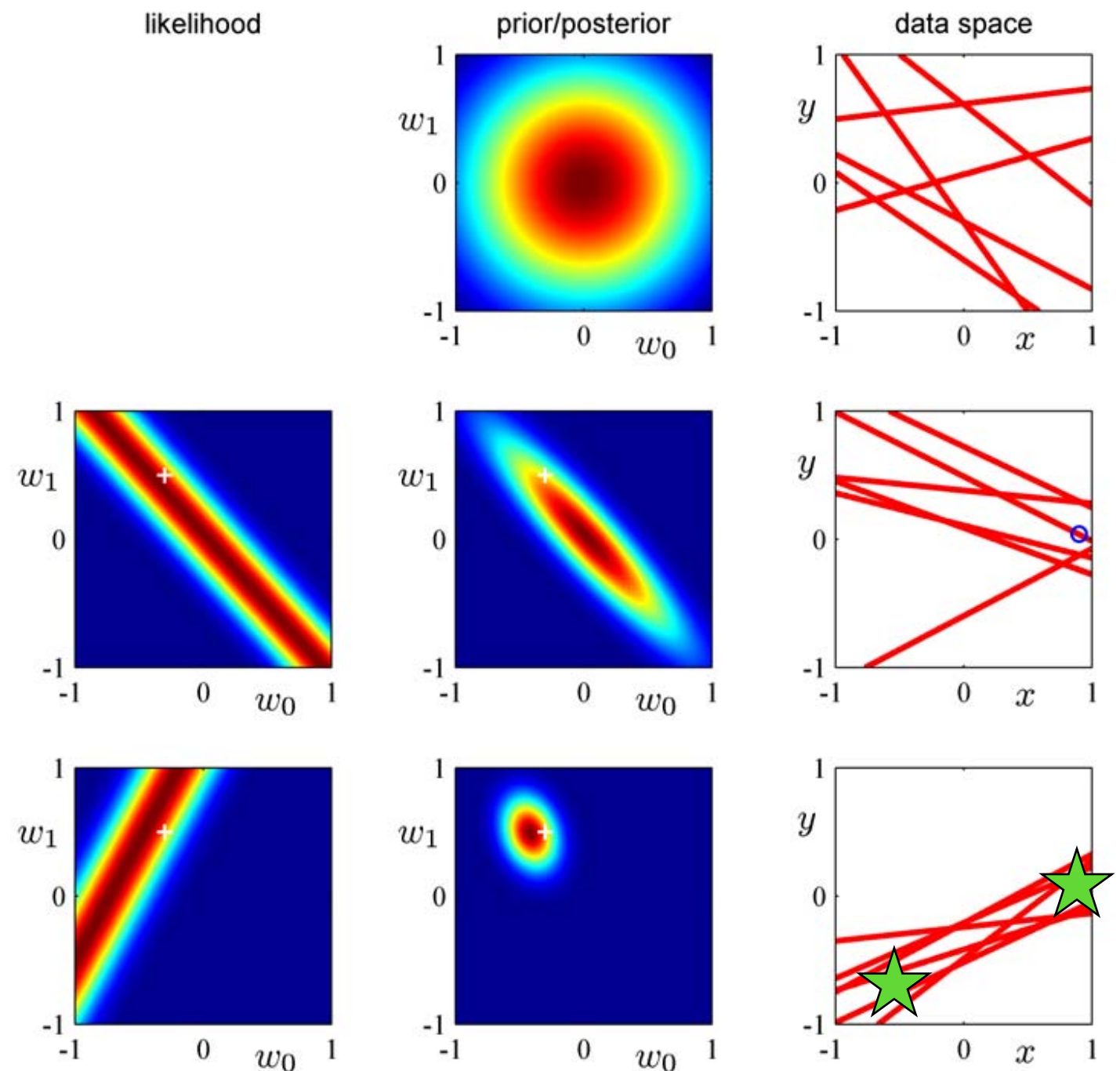


Figure: Sequential Bayesian learning (Bishop 3.7)

$$p(\mathbf{w} | (x_1, t_1), (x_2, t_2), \alpha, \beta) \propto p(t_2 | x_2, \mathbf{w}, \beta) p(\mathbf{w} | (x_1, t_1), \alpha, \beta)$$

Example: Sequential Bayesian Learning

- After 19 data points

$$D = \{(x_i, t_i)\}_{i=1}^{19}$$

- Prior

$$p(\mathbf{w} | \{(x_n, t_n)\}_{n=1}^{19}, \alpha, \beta)$$

- Likelihood

$$p(t_{20} | x_{20}, \mathbf{w}, \beta)$$

$$N \rightarrow \infty$$

$\Rightarrow$ very sharp posterior!

- Posterior

$$p(\mathbf{w} | \{(x_n, t_n)\}_{n=1}^{20}, \alpha, \beta) \propto p(t_{20} | x_{20}, \mathbf{w}, \beta) p(\mathbf{w} | \{(x_n, t_n)\}_{n=1}^{19}, \alpha, \beta)$$

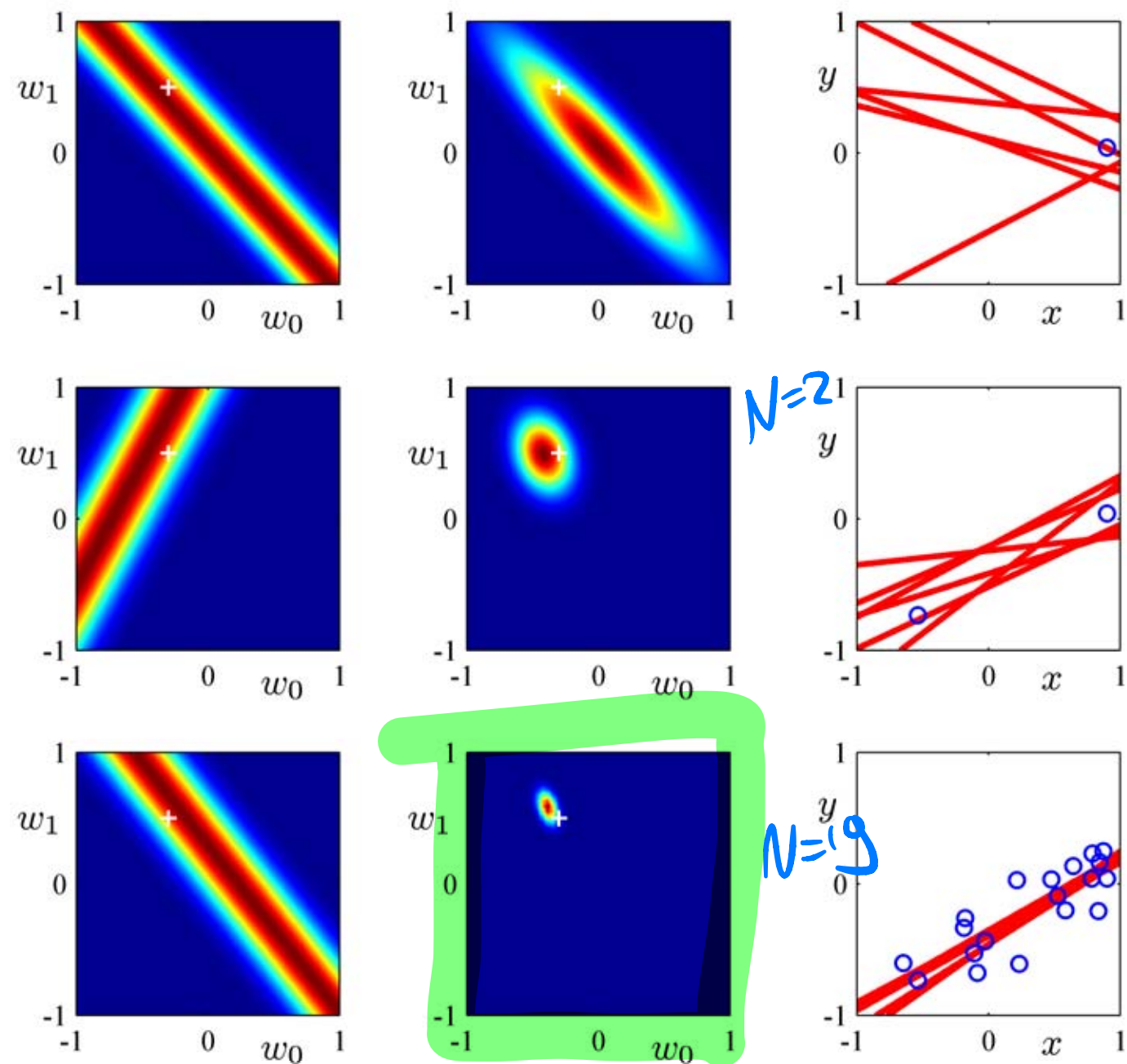


Figure: Sequential Bayesian learning (Bishop 3.7)

Bayesian Linear Regression

- Limiting cases of the posterior

- $p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \alpha, \beta) = \mathcal{N}(\mathbf{w} | \mathbf{m}_N, \mathbf{S}_N)$

with $\mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{t}$ $\mathbb{R}^{N \times M}$

$$\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \underbrace{\Phi^T \Phi}_{\text{gets large}}$$

- After infinite amount of data ($N \rightarrow \infty$):

- $\lim_{N \rightarrow \infty} [\Phi^T \Phi]_{ij} = \lim_{N \rightarrow \infty} \sum_{n=1}^N \phi_i(\mathbf{x}_n) \phi_j(\mathbf{x}_n) \propto N$

(prop or normal to N)

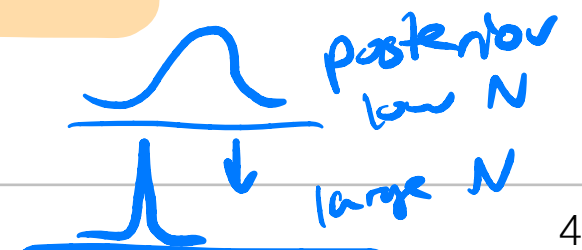
- $\lim_{N \rightarrow \infty} \mathbf{S}_N = \mathbf{0}$ zero matrix

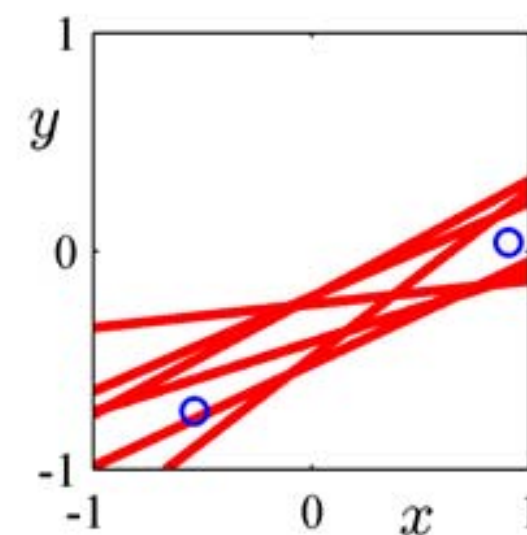
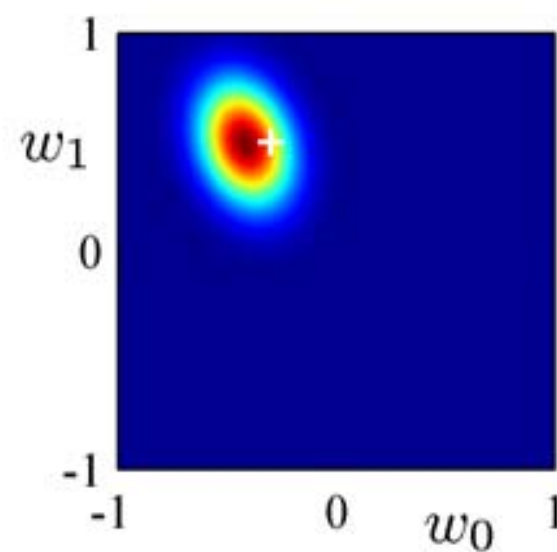
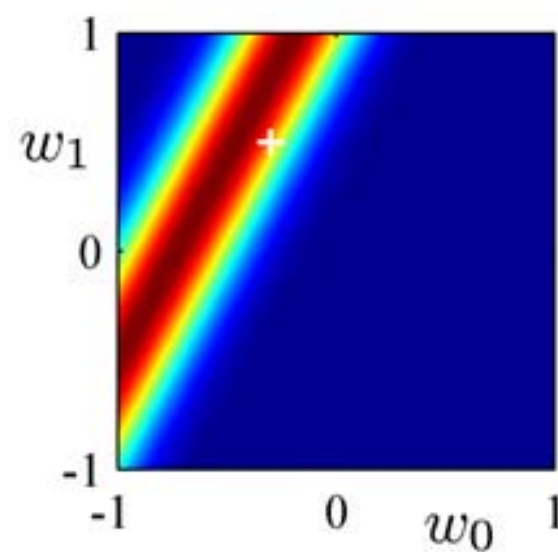
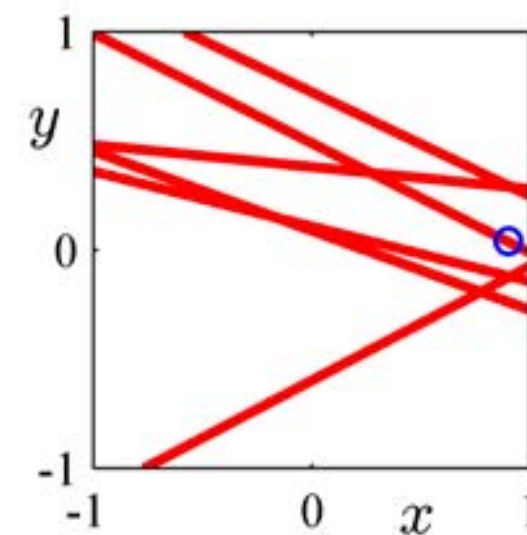
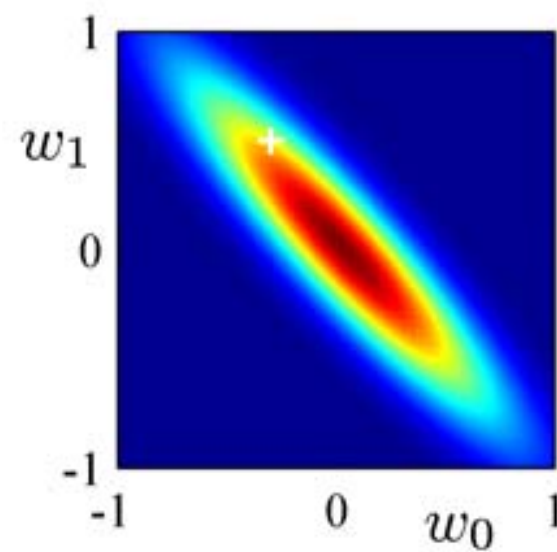
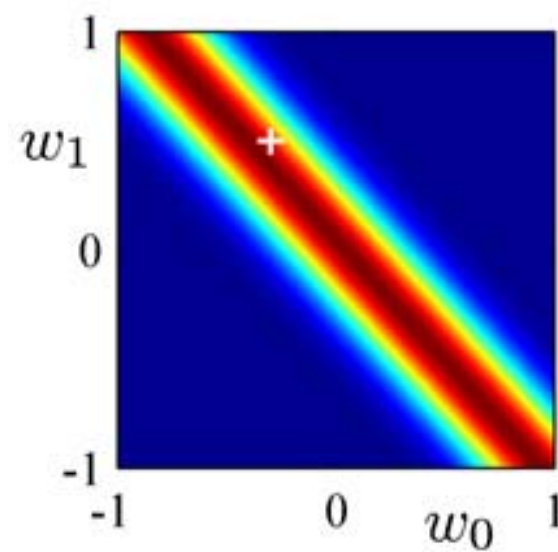
- $\lim_{N \rightarrow \infty} \mathbf{m}_N = \lim_{N \rightarrow \infty} \beta (\cancel{\alpha \mathbf{I}} + \cancel{\beta} \Phi^T \Phi)^{-1} \Phi^T \mathbf{t} = (\Phi^T \Phi)^{-1} \Phi^T \mathbf{t}$

In the limit $N \rightarrow \infty$: $ML = MAP = \text{Bayesian}$

Since prior plays no role

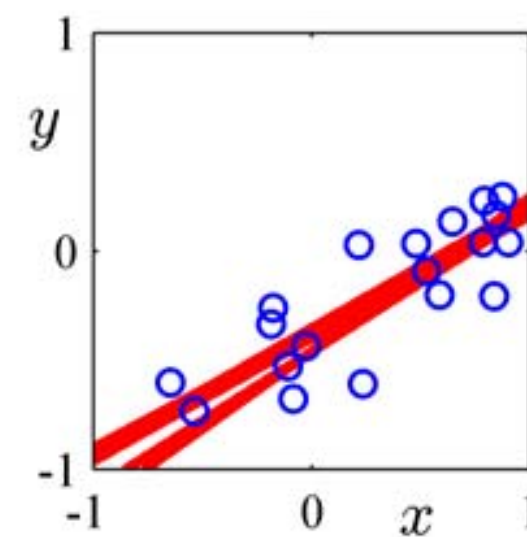
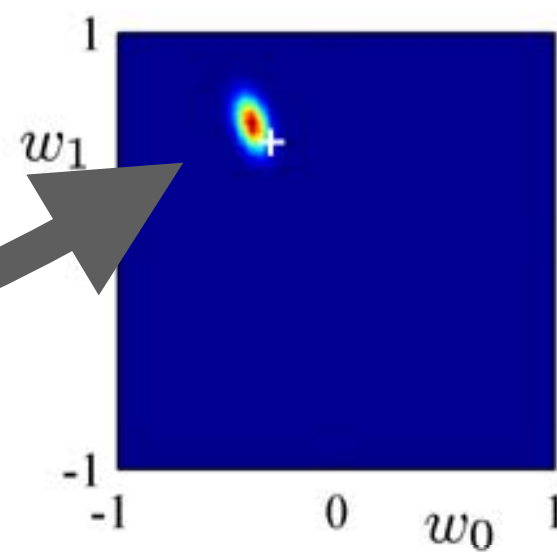
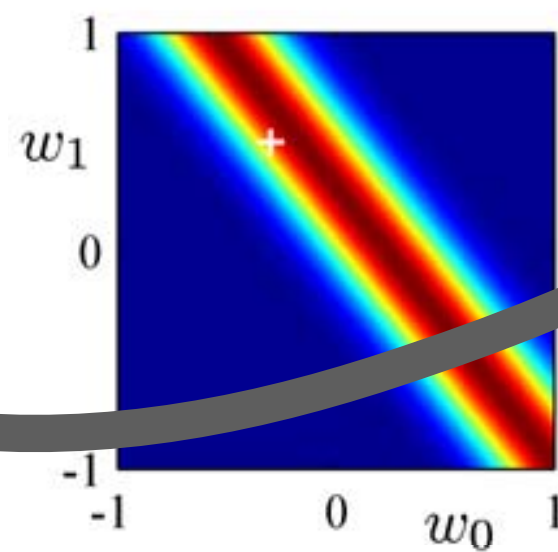
Since posterior is sharp





$N \rightarrow \infty$

ML solution



Bayesian Linear Regression

- ▶ Important limits

- ▶ If $N \rightarrow \infty$

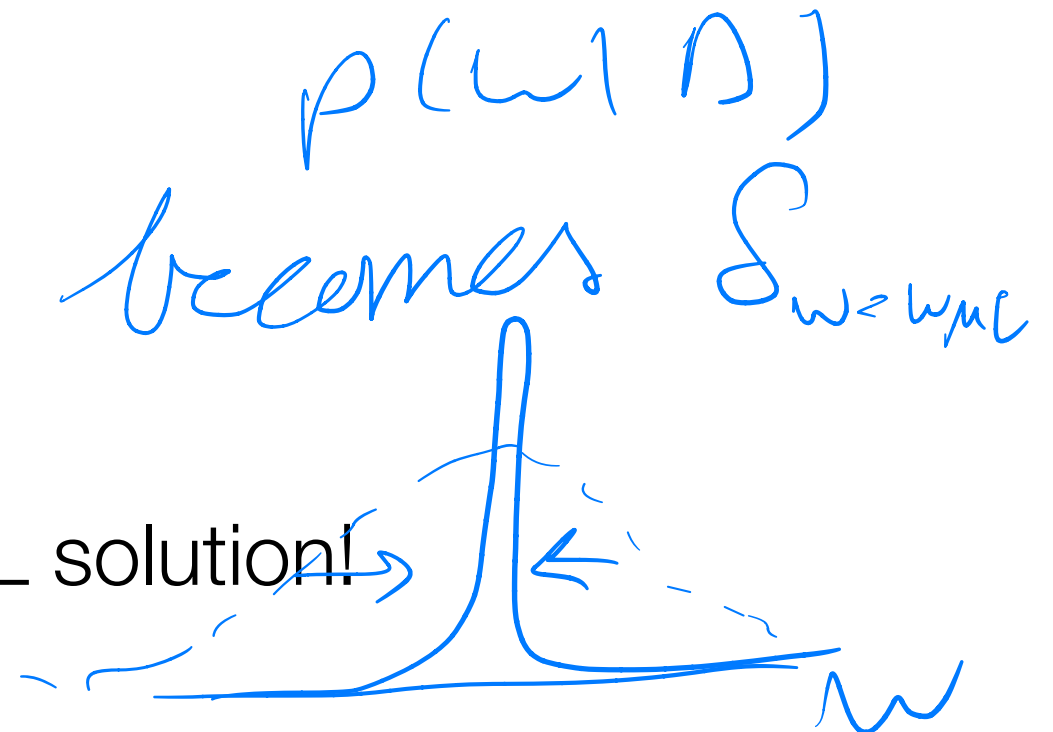
- ▶ MAP solution converges to ML solution!

- ▶ Bayesian predictive distribution

$$p(t' | x', D) = \int p(t' | x', \mathbf{w}) p(\mathbf{w} | D) d\mathbf{w}$$

converges to?

$$p(t' | x', w_{ML})$$





Machine Learning 1

Lecture 4.5 - Supervised Learning
Bayesian Linear Regression - **Predictive
Distribution**

Erik Bekkers

(Bishop 3.3.2)



Predictive Distribution

- ▶ Observed dataset with inputs $\mathbf{x} = (\mathbf{x}_1, \dots, \mathbf{x}_N)^T$ and targets $\mathbf{t} = (t_1, \dots, t_N)^T$
- ▶ Gaussian Posterior distribution (from Gaussian prior and Gaussian likelihood)

$$\begin{aligned} \text{▶ } p(\mathbf{w} | \mathbf{X}, \mathbf{t}, \alpha, \beta) &= \mathcal{N}(\mathbf{w} | \mathbf{m}_N, \mathbf{S}_N) && \text{with } \mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{t} \\ & && \mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi \end{aligned}$$

- ▶ Parametrized Gaussian predictive distribution:

$$\text{▶ } p(t' | \mathbf{x}', \mathbf{w}, \beta) = \mathcal{N}(t' | \phi(\mathbf{x}')^T \mathbf{w}, \beta^{-1})$$

- ▶ Gaussian Bayesian predictive distribution for new input

$$\begin{aligned} \text{▶ } p(t' | x', \mathbf{X}, \mathbf{t}, \alpha, \beta) &= \int \mathcal{N}(t' | \phi(\mathbf{x}')^T \mathbf{w}, \beta^{-1}) \mathcal{N}(\mathbf{w} | \mathbf{m}_N, \mathbf{S}_N) d\mathbf{w} \\ &= \mathcal{N}(t' | \phi(\mathbf{x}')^T \mathbf{m}_N, \sigma_N^2(\mathbf{x}')) \end{aligned}$$

Bishop Eq.
2.115

- ▶ With $\sigma_N^2(\mathbf{x}') = \beta^{-1} + \phi(\mathbf{x}')^T \mathbf{S}_N \phi(\mathbf{x}')$

Predictive Distribution

- Datasets:
 - $t = \sin(2\pi x) + \epsilon$
 - $\epsilon \sim \mathcal{N}(0, \beta^{-1})$
- Dataset sizes:
 - $N = 1, 2, 4, 25$
- Model:
 - $y(x, \mathbf{w}) = \boldsymbol{\phi}(x)^T \mathbf{w}$
 - $\phi_j(x)$: Gaussian basis functions

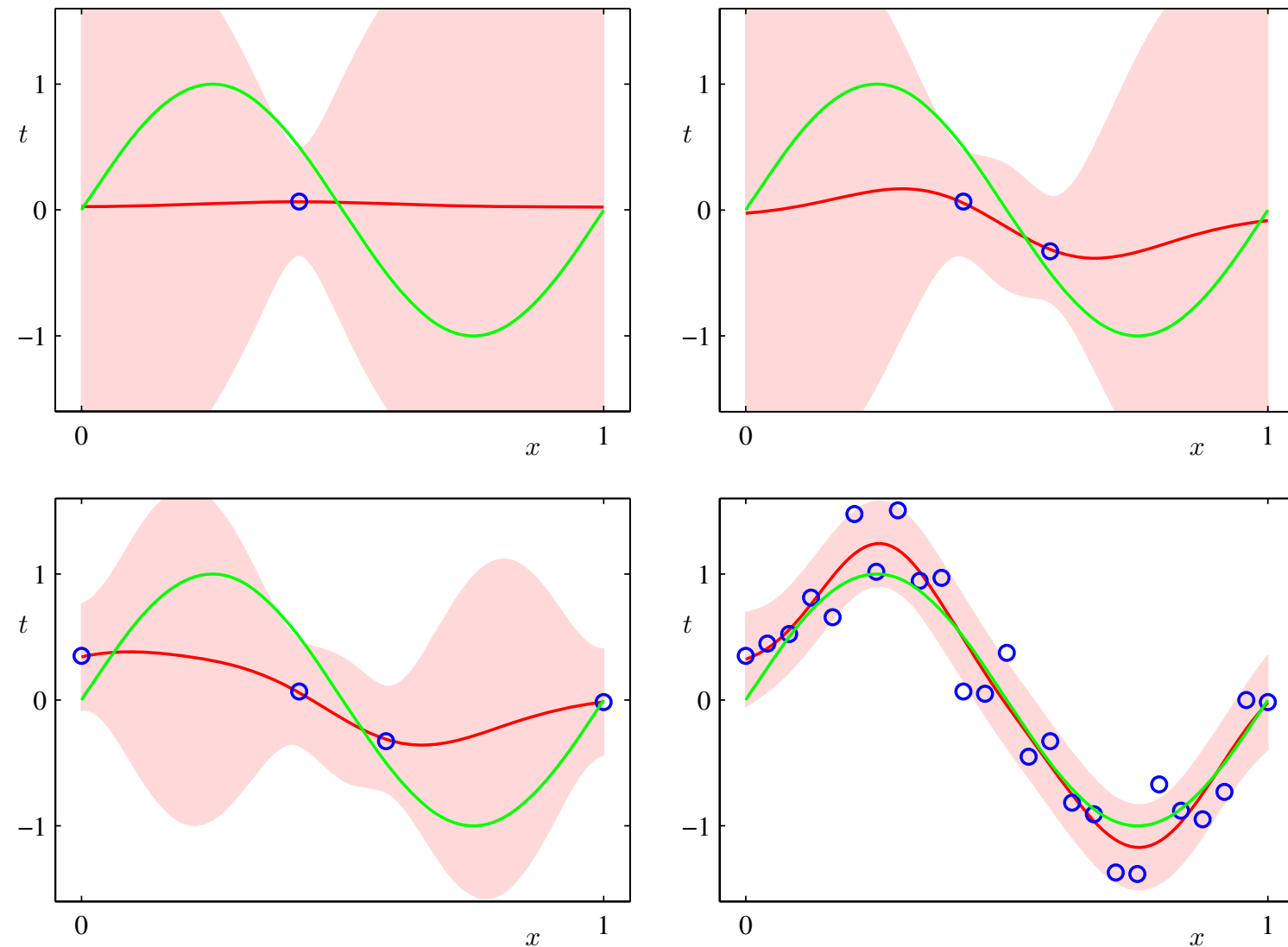


Figure: Predictive distribution (Bishop 3.8)

- Predictive distribution:
 - $p(t' | x', \mathbf{X}, \mathbf{t}, \alpha, \beta) = \mathcal{N}(t' | \boldsymbol{\phi}(\mathbf{x}')^T \mathbf{m}_N, \sigma_N^2(\mathbf{x}'))$
 - $\sigma_N^2(\mathbf{x}') = \beta^{-1} + \boldsymbol{\phi}(\mathbf{x}')^T \mathbf{S}_N \boldsymbol{\phi}(\mathbf{x}'), \quad \mathbf{m}_N = \beta \mathbf{S}_N \boldsymbol{\Phi}^T \mathbf{t}, \quad \mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \boldsymbol{\Phi}^T \boldsymbol{\Phi}$

Predictive Distribution

▶ Datasets:

▶ 1. The Bayesian predictive distributions are

▶ $\epsilon \sim \mathcal{N}(0, \sigma^2)$ Gaussian (with analytic expressions based on prior parameters and data matrix Φ)

▶ Dataset sizes:

▶ $N = 1, 2, 4, 25$
2. The Bayesian model gets **more certain close to previously seen data**

▶ Model:

▶ $y(x, \mathbf{w}) = \phi(x)^T \mathbf{w}$

▶ $\phi_j(x)$: Gaussian basis functions
3. The Bayesian model overall **gets more certain with more data**

▶ Predictive distribution:

▶ 4. The (mean) predictions themselves **get more accurate with more data**

▶ $\sigma_N^2(\mathbf{x}') = \beta^{-1} + \phi(\mathbf{x}')^T \mathbf{S}_N \phi(\mathbf{x}')$, $\mathbf{m}_N = \beta \mathbf{S}_N \Phi^T \mathbf{t}$, $\mathbf{S}_N^{-1} = \alpha \mathbf{I} + \beta \Phi^T \Phi$

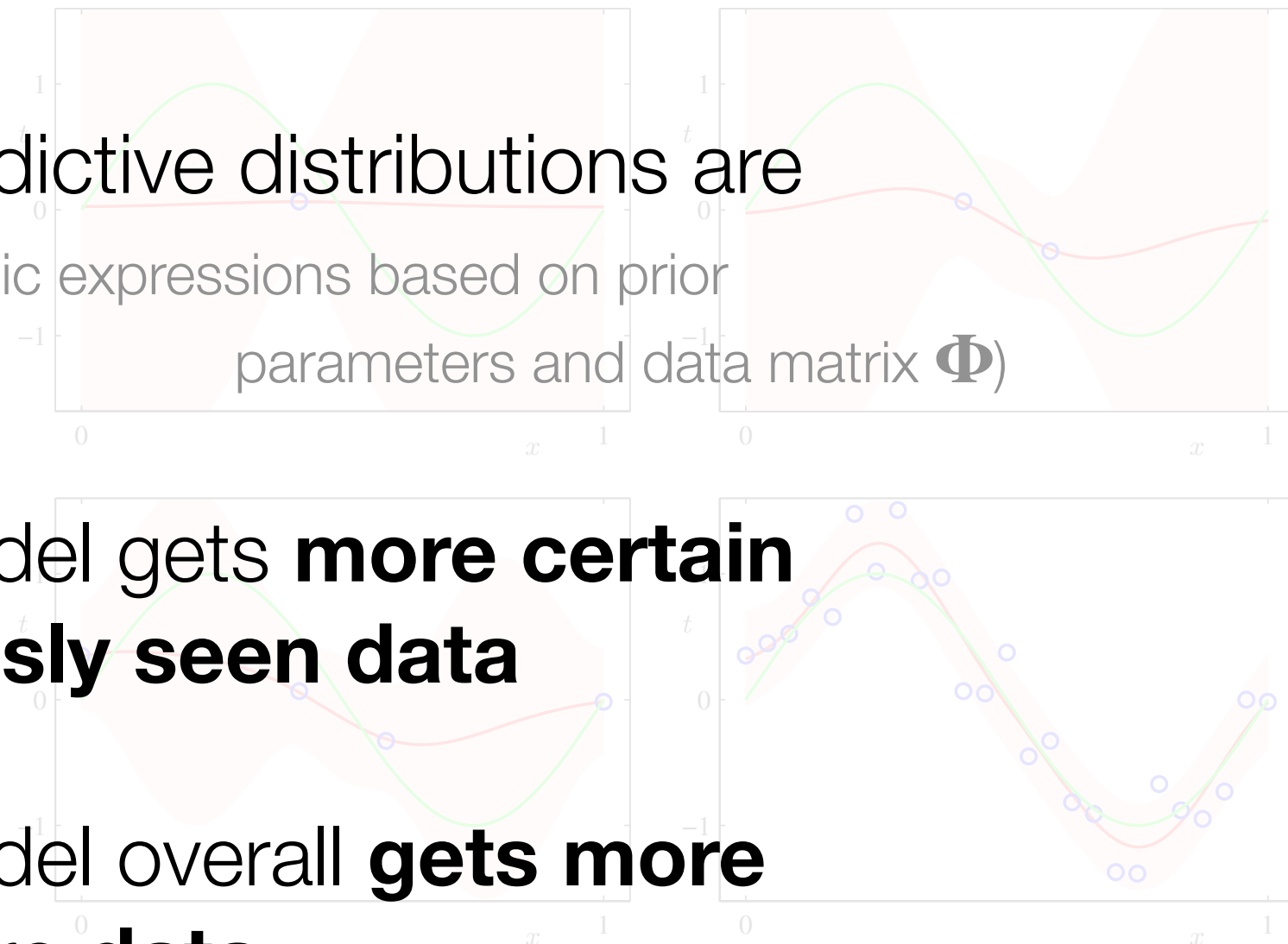


Figure: Predictive distribution (Bishop 3.8)



Machine Learning 1

Lecture 5.2 - Supervised Learning
Bayesian Linear Regression - Bayesian Model
Comparison

Erik Bekkers

(Bishop 3.4)





Machine Learning 1

Lecture 5.3 - Supervised Learning
Bayesian Linear Regression - Approximating
the Model Evidence

Erik Bekkers

(Bishop 3.5.0)





Machine Learning 1

Lecture 5.4 - Supervised Learning

Classification - Decision Regions

Erik Bekkers

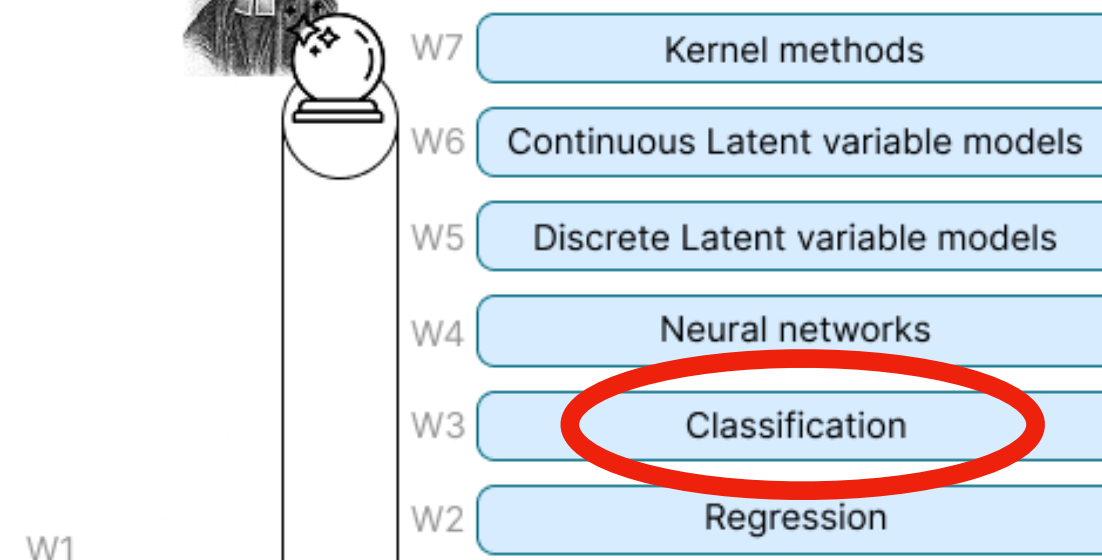
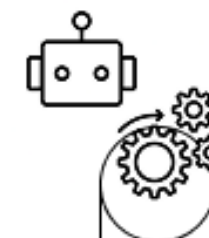
(Bishop 1.5, 4.1)

Machine Learning 1

The probabilistic
view



The algorithmic
view



Probabilistic modeling

Random variables,
distributions, maximum
likelihood, MAP, ...

Optimization

Exact/analytic, (stochastic)
gradient descent, constraint
optimization, 2nd order

Mathematical tools

Multi-variate calculus,
linear algebra

Classification through decision regions

▶ Input: $\mathbf{x} = (x_1, \dots, x_D)^T \in \mathbb{R}^D$

▶ Target:

▶ 2-class targets:

$t \in \{C_1, C_2\} = \{\text{"cat"}, \text{"dog"}\}$

▶ Multi-class targets

$t \in \{C_1, C_2, \dots, C_K\}$

$$\underline{t} = \begin{pmatrix} 0 \\ 0 \\ \vdots \\ 1 \end{pmatrix}$$

$$\underline{t} = \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix} \in \mathbb{R}^K$$

$$\underline{t} = \begin{pmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$$

Strategy:

One-hot encoding

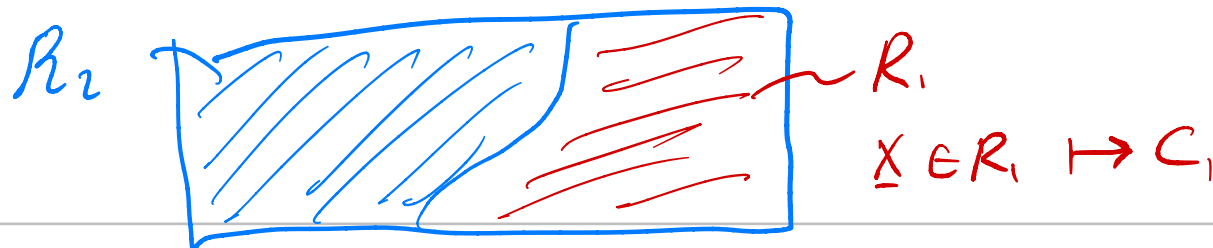
a vector with only a 1 at the class-index but all other elements zero

▶ Divide input space $\mathbb{R}^D$ into K decision regions R_k

▶ Assign each decision region to a class C_k

▶ Boundaries of decision regions are called *decision boundaries/surfaces*.

if $\underline{x}$ falls in R_2
 ($\underline{x} \in R_2$) then
 classify as C_2 ($\underline{x} \mapsto C_2$)
 "maps to"



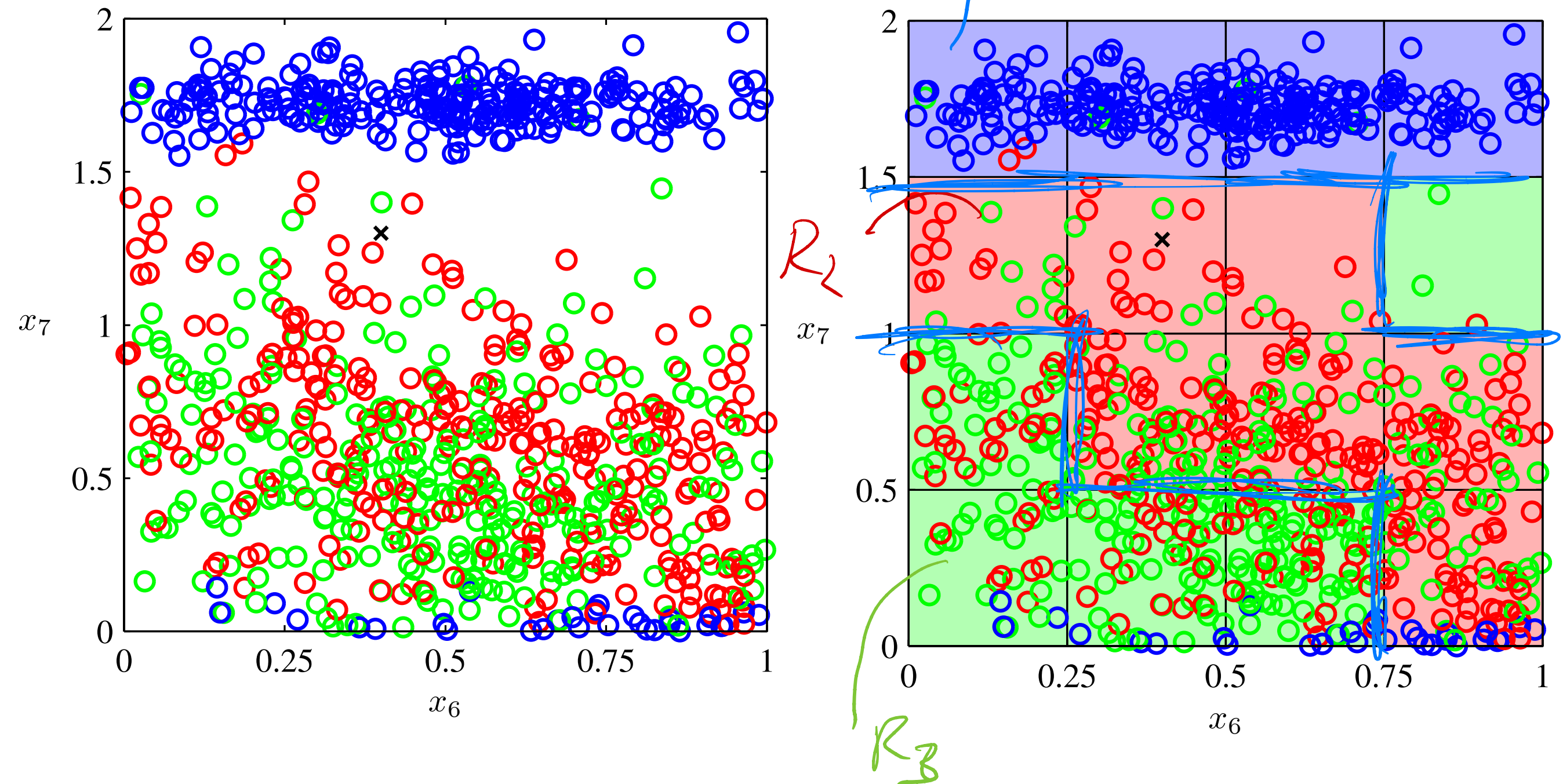
$$\underline{t} = \begin{pmatrix} t_1 \\ t_2 \\ \vdots \\ t_k \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ \vdots \end{pmatrix} \in \{0, 1\}^k$$

is a one-hot encoding for class C_k if $t_i = 1$ if $i = k$ and zero otherwise.

In the above $\underline{t}$ is a one-hot encoding for class C_2 since $t_2 = 1$ and all other entries are zero.

Such a one-hot encoding can be thought of as a categorical distribution, assigning probability 1 to the corresponding class and zero to others $p(\text{class} = C_k) = 1$.

Classification through Decision Regions



Figures: 3 class problem with decision boundaries. (Bishop 1.19 & 1.20)

Linear Classification

- ▶ Linear Classification: consider only *linear* decision boundaries
- ▶ For D -dimensional input space: $\mathbf{x} \in \mathbb{R}^D$
decision surface is a $D - 1$ dimensional hyperplane
- ▶ Datasets whose classes can be separated *exactly* by linear decision surfaces are called **linearly separable**

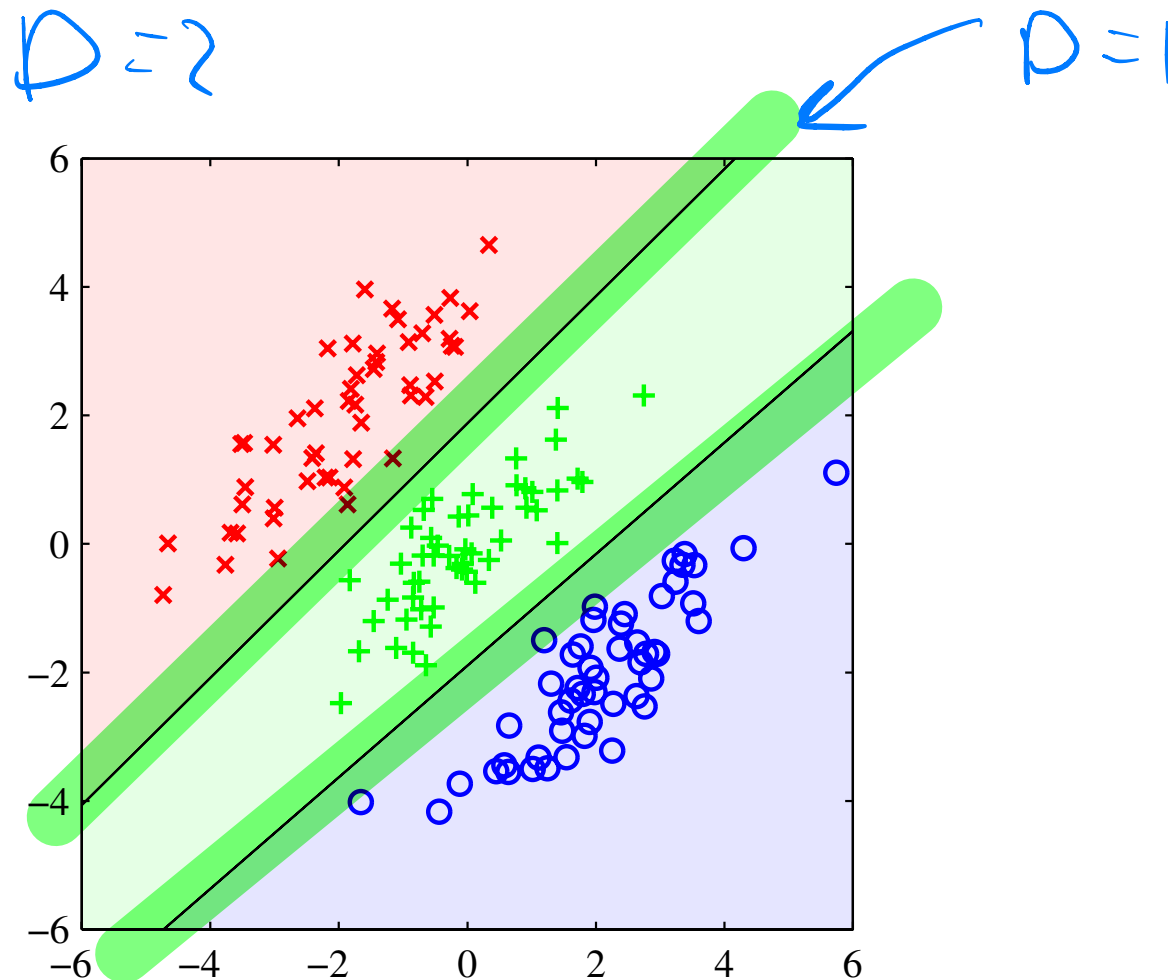


Figure: Linearly separable dataset (Bishop 4.5)

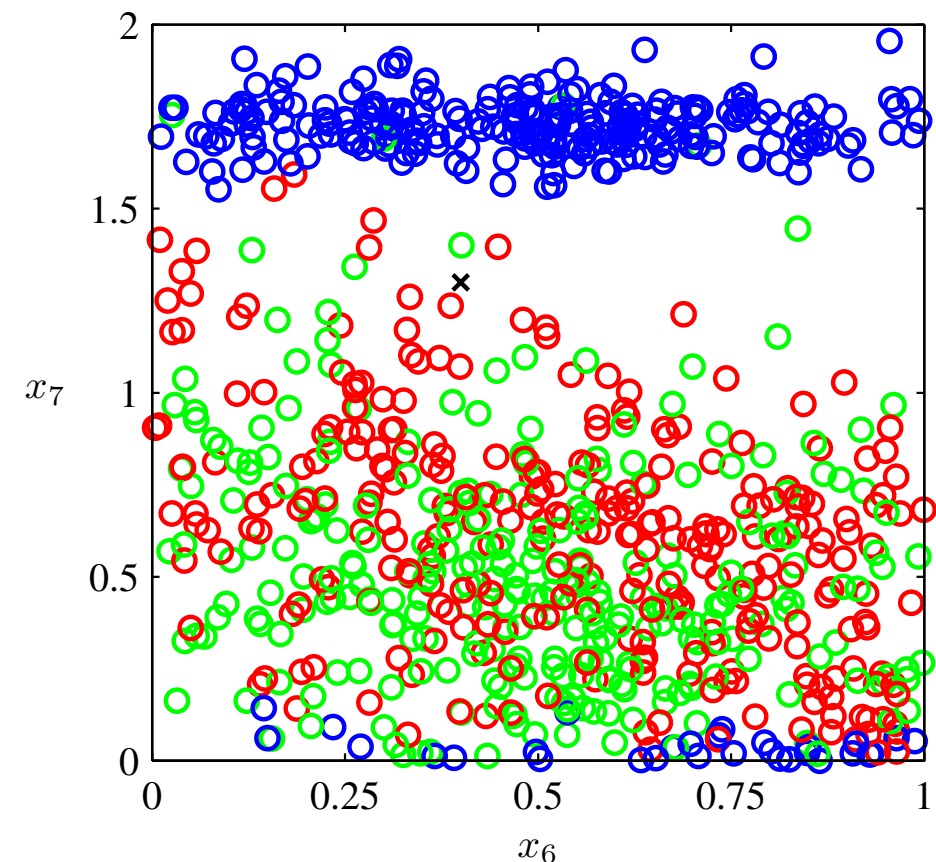


Figure: Not linearly separable dataset (Bishop 1.19)

Multiple Classes ($K > 2$)

Binary classifiers!

- ▶ $K = 2$ classes:
- ▶ 1 classifier determines
- ▶ Multiple classes: $K > 2$
- ▶ $K - 1$ classifiers:

▶ One-versus-the-rest

using multiple binary classifiers

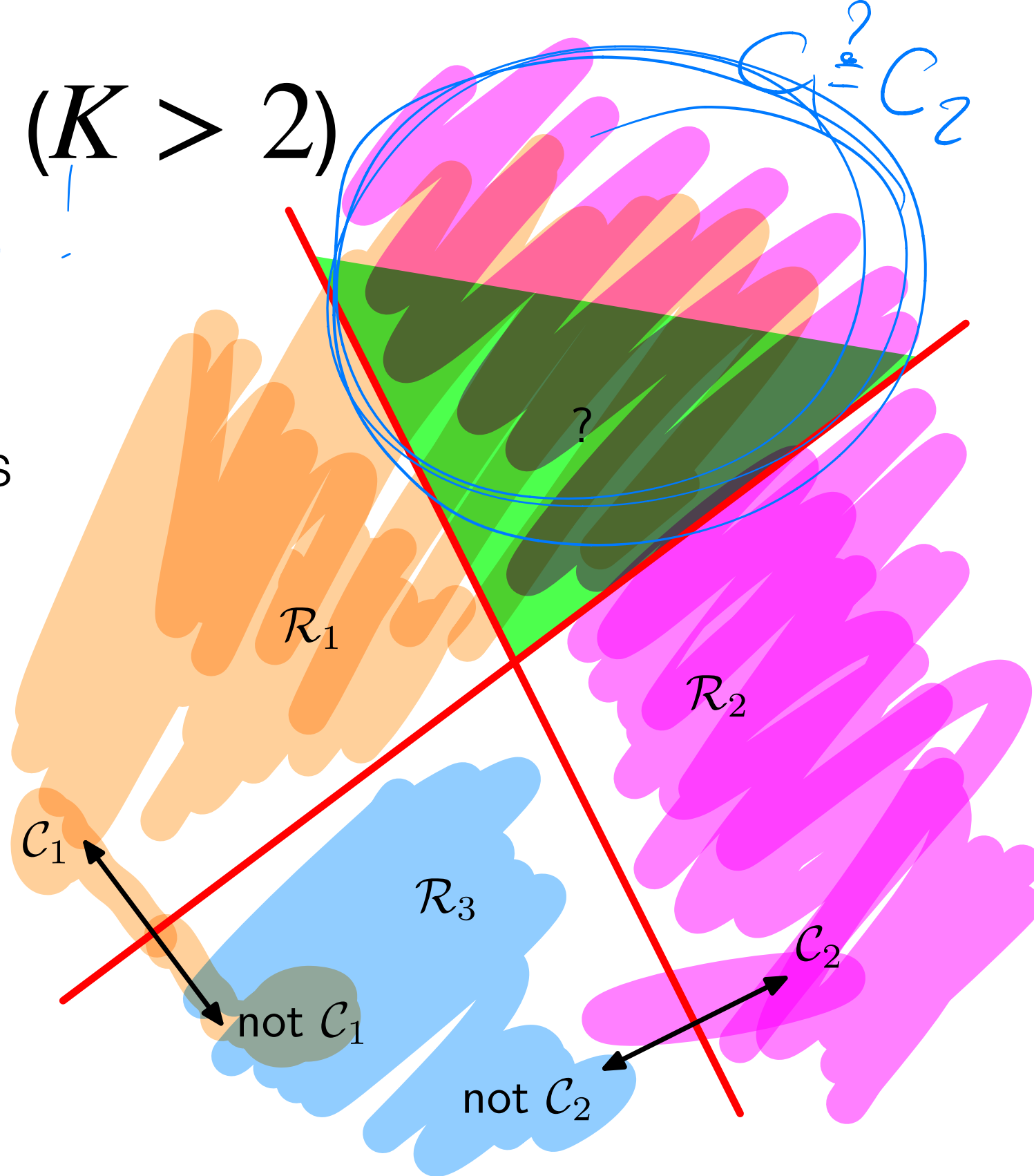
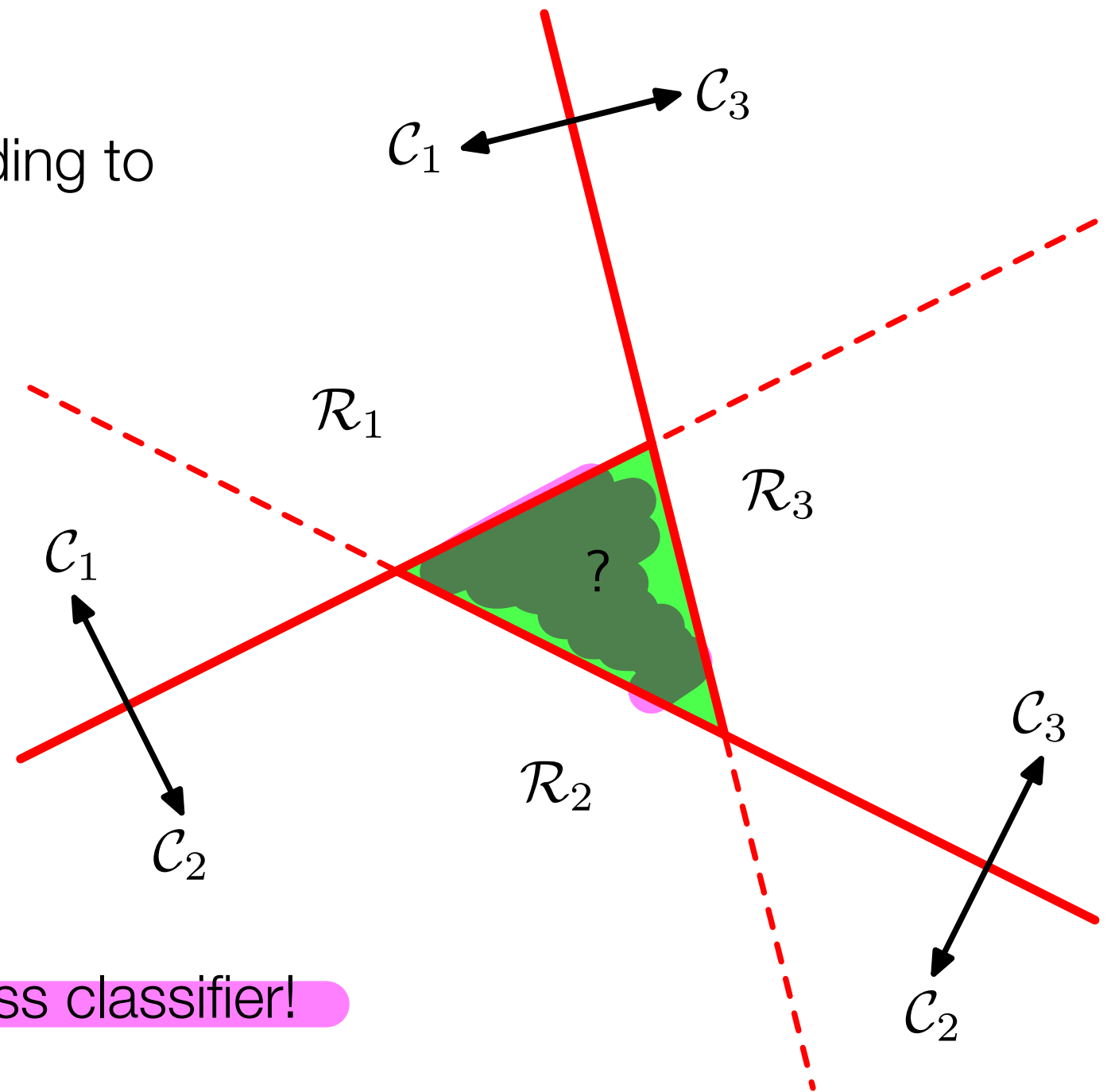


Figure: one-versus-the-rest classifiers (Bishop 4.2)

Multiple Classes ($K > 2$)

- ▶ $K(K - 1)/2$ classifiers:
- ▶ Points are classified according to majority vote of classifiers
- ▶ one-versus-one



- ▶ **Solution:** Make one K -class classifier!
(See later)

Figure: one-versus-one classifiers (Bishop 4.2)

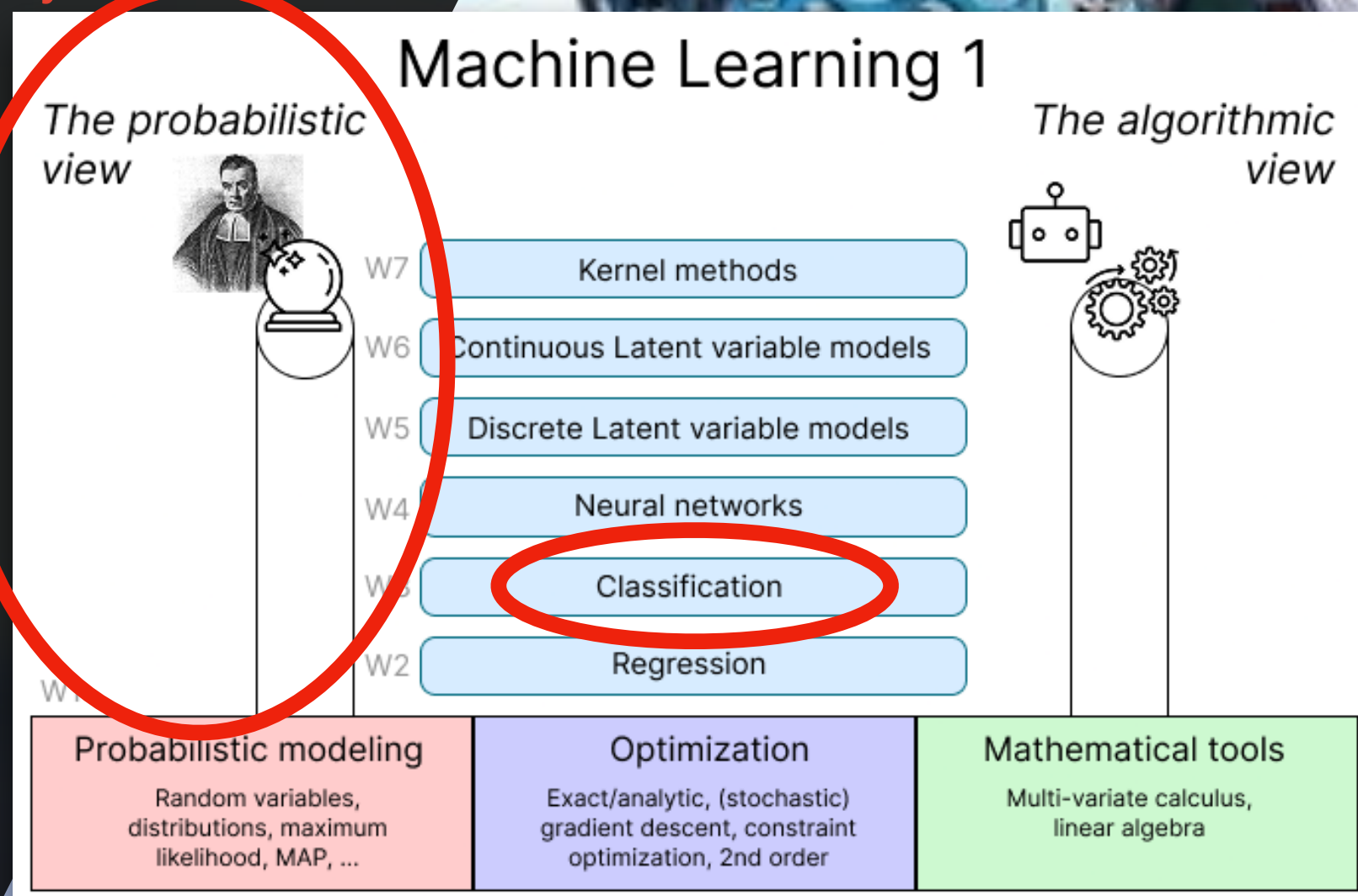


Machine Learning 1

Lecture 5.5 - Supervised Learning
Classification - **Decision Theory**

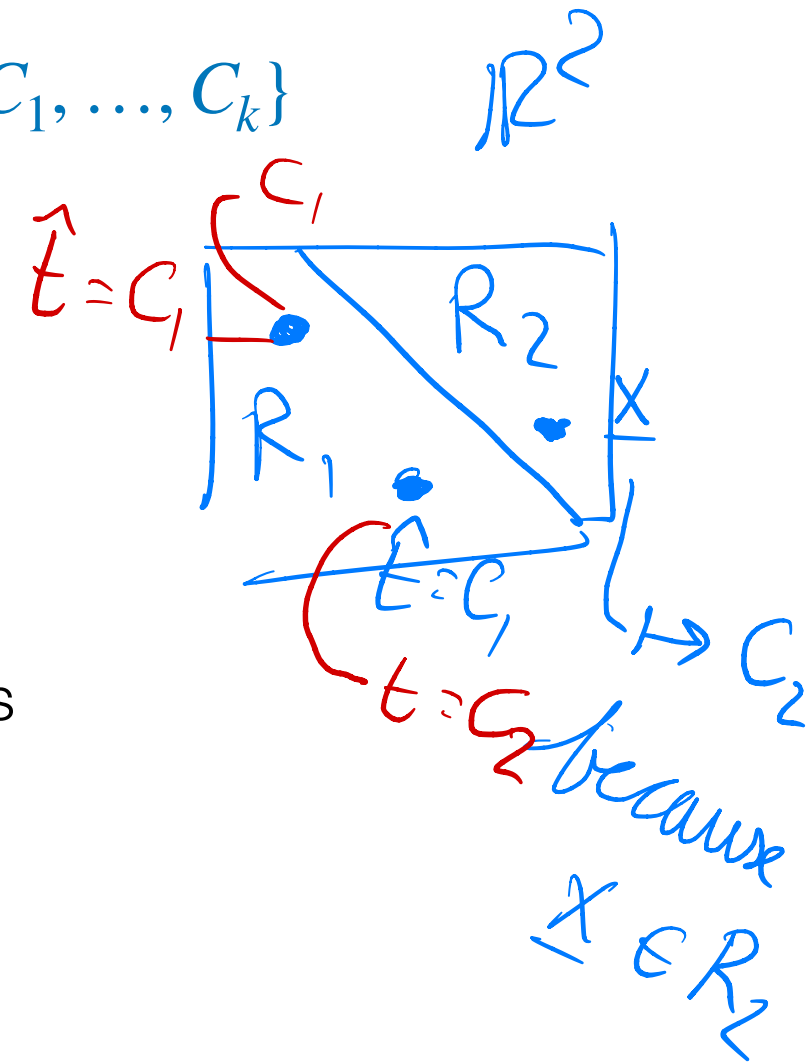
Erik Bekkers

(Bishop 1.5)



Decision theory

- Dataset: Input vectors $\mathbf{x} \in \mathbb{R}^D$, ground truth targets $t \in \{C_1, \dots, C_k\}$
- Divide input space $\mathbb{R}^D$ into K decision regions R_k
- Every observed datapoint: $\begin{cases} \text{ground truth} \\ \text{prediction} \end{cases}$
- Confusion matrix: ground truth classes vs. predicted classes



matrix of counts or frequencies for each type of prediction

ground truth ↓

	R_1	R_2	...	R_K
C_1	6	1	...	0
C_2	5	3	...	1
$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$
C_K	2	0	...	8

- Diagonal elements: correctly classified
- Off-diagonal elements: misclassified

Decision theory: Misclassification Rate

- Classification goal: Minimize the misclassification rate

- Assume observations are drawn from joint distribution $(\mathbf{x}, t) \sim p(\mathbf{x}, t)$

- Probability of a misclassification:

$$\underline{p(\text{correct}) + p(\text{mistake}) = 1}$$

Note: For every one of K classes, we can make $K - 1$ mistakes (so $K(K - 1)$ mistakes)
(Only when predicted class C_i coincides with true class C_k)

for all labels
or
predicted

$$p(\text{mistake}) = \sum_{i=1}^K \sum_{k \neq i} p(\mathbf{x} \in R_i, t = C_k)$$

assign to class i
should belong to class k

$$= 1 - p(\text{correct}) = 1 - \sum_{k=1}^K p(\mathbf{x} \in R_k, C_k)$$

Minimizing misclassification rate

- Assign $\mathbf{x}$ to class C_k if

$$p(\mathbf{x}, t = C_k) > p(\mathbf{x}, t = C_j)$$

- Note: $p(\mathbf{x}, C_k) = p(C_k | \mathbf{x})p(\mathbf{x})$

instead maximize:

$$p(C_k | \mathbf{x}) > p(C_j | \mathbf{x})$$

$$\hat{C}(\mathbf{x}) = \underset{C \in \{C_1, C_2, \dots, C_K\}}{\operatorname{argmax}} p(C | \mathbf{x})$$

Decision theory: Misclassification Rate

$\hat{x}$: Decision boundary

x_0 : Optimal Decision boundary

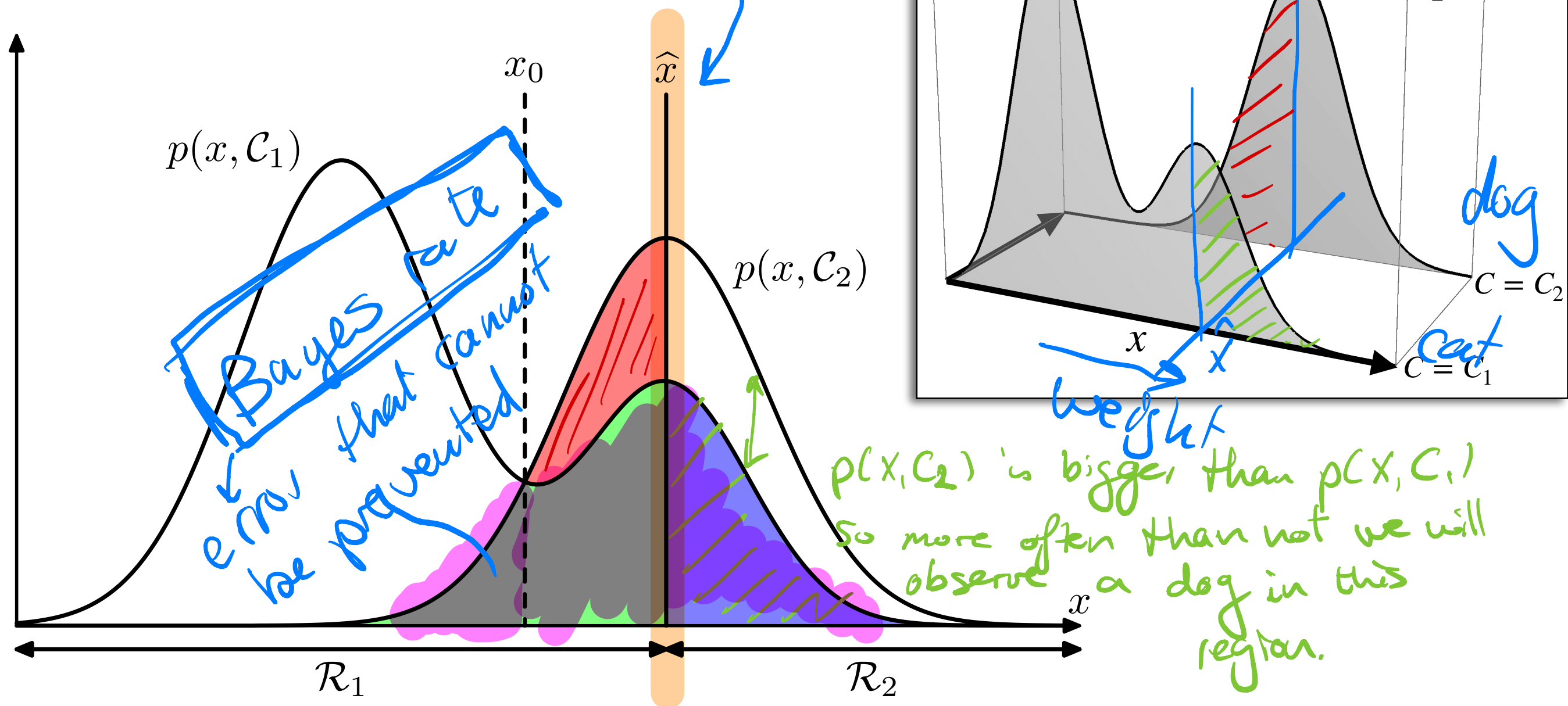
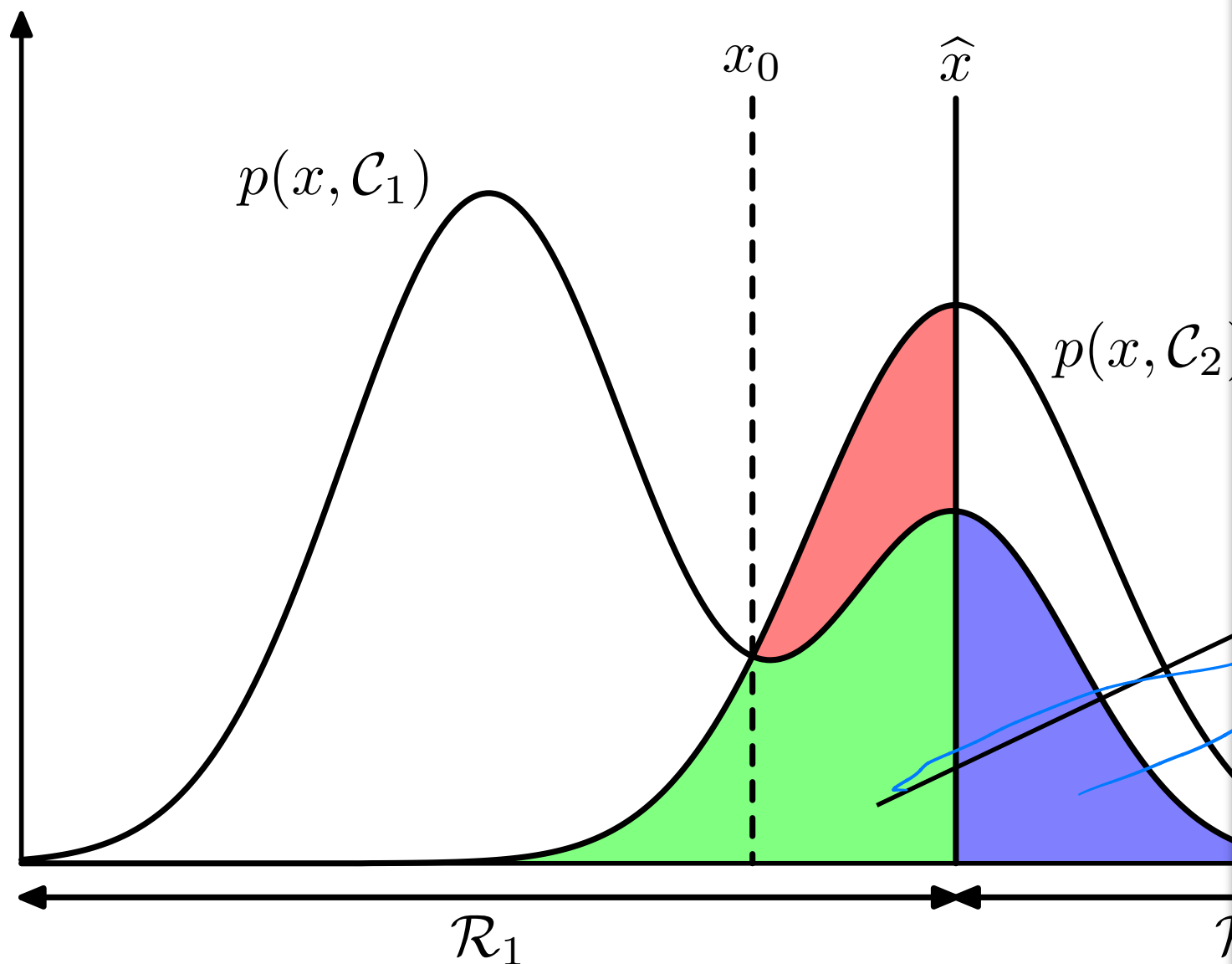


Figure: joint probability distributions and decision boundary (Bishop 1.24)

Decision theory: Misclassification Rate

$\hat{x}$: Decision boundary

x_0 : Optimal Decision boundary



Bayes Classifier

$$\hat{C}(x) = \operatorname{argmax}_k p(C_k, x)$$

$$= \operatorname{argmax}_k p(C_k | x) p(x)$$

Bayes Rate

$$\int (1 - \max_k p(C_k | x)) p(x) dx$$

Figure: joint probability distributions and decision boundary (Bishop 1.24)

Minimizing the Misclassification Rate: Problems

- ▶ Not all errors have the same impact!

Example: Medical diagnosis of cancer

- ▶ Error 1: Label a healthy person as having cancer.
- ▶ Error 2: Label a sick person as healthy. Lack of treatment!

important

severity of error

- ▶ If cancer only occurs in 1% of all patients, a classifier which labels everyone as healthy has a misclassification rate of 1%!

class imbalance

should also be corrected for

Expected Loss

$$p(x \in (a, b)) = \int_a^b p(x) dx$$

$$R, a \leq x \leq b$$



- Possible solution: use different weights for different error types

$$L = \begin{pmatrix} \text{label cancer} & \text{label healthy} \\ 0 & 1000 \\ 1 & 0 \end{pmatrix} \begin{matrix} \text{true cancer} \\ \text{true healthy} \end{matrix}$$

- Expected loss:

$$\mathbb{E}_{(\mathbf{x}, t) \sim p(\mathbf{x}, t)}[L] = \sum_{k, j} L_{kj} p(x \in R_j, t = C_k) = \sum_{k, j} L_{kj} \int_{R_j} p(x, C_k) dx$$

- Objective: Minimize the expected loss:

$$\text{Assign } \mathbf{x} \text{ to } C_k \text{ if } \sum_{j=1}^K L_{jk} p(\mathbf{x}, C_j) \text{ is minimal}$$

Classification Strategies

- Discriminant functions

Direct mapping of input to target:

$$\hat{t} = y(\underline{x}, \underline{w})$$

- Probabilistic discriminative models

Posterior class probabilities:

$$p(C_k | \underline{x})$$

- Probabilistic generative models

Class-conditional densities:

$$\left. \begin{array}{l} p(\underline{x} | C_k) \\ p(C_k) \end{array} \right\} \begin{array}{l} p(\underline{x}, C_k) \\ p(C_k | \underline{x}) \end{array}$$

Prior class probabilities: